



Re: Revisiting 2021 Computer Vision Project for Possible Publication

From D.M. Fisher <dmf41@cantab.ac.uk>

Date Tue 7/7/2026 3:06 PM

To Chris Town <cpt23@cam.ac.uk>

Hi Chris,

Thank you for the rapid response on the Columbia application! I have submitted my application so hopefully that works out! I'll keep you posted on any updates on that front.

Thank you again for all of the feedback! I have made the recommended corrections to my abstract and introduction sections, and am still making my way through the methodology feedback.

At the same time, I have been focusing on finalizing the code for our final figures and data. I have implemented the remaining TODO's we specified for v7.1, the error decomposition adapted from v5.0 to v7.2 (incorporating the various improvements since we originally did the ablation), and v7.3 showing the boundary encoding ablation, similarly incorporating improvements like I did in v7.2. I will elaborate more on these below along with addressing the areas you recommended prioritizing:

1. The Bugs Bunny S/L ratio check:

I added some additional book-keeping to track and display S/L ratio and tested it for several images to include bugs bunny. For bugs bunny, all 41 surviving regions encode via the spline path, with zero polyline fallbacks. The segmentation initially identified 1,532 regions before merging down to 41, so the merge step is aggressive, but the survivors are all spline-encoded.

To me this means the near-identical PSNR between conditions C and F reflects actual spline fidelity rather than an artifact of the fallback path and points to the dominant error source remaining the interior appearance model, consistent with the large gap between conditions B and F. In fact, through all of my testing thus far, I have not identified any region for any image that is falling back to polyline representation, which is noteworthy.

2. The minimum v7.1 codec:

I implemented region-hybrid DCT encoding and incorporated your earlier suggestion of making it 3 tiered with a linear, quadratic parametric, and linear+DCT encoding. For debugging purposes, I still have it print to imComp.txt (except the DCT encoded regions). I noted when implementing the hybrid DCT that I need a really small lambda value to get even 10% of the regions DCT encoded, which tells me that even when I barely penalize rate, quadratic encoding is still chosen for most regions, which I think is a finding worth capturing. To denote which type of encoding a region follows, I incorporated an additional flag in the "M" encoding line.

After the hybrid DCT, I implemented the full encoding pipeline. I started by shifting from floating point to fixed quantized representation. This exposed an error that when I stored the planar model, the "a" value (intercept) is always a starting point relative to the top left corner of the image and would therefore be a

massive negative number in a lot of cases, so I changed it to store the centroid of each cluster and interpret relative to that.

I incorporated delta coding for S and L. Since the "M" planar color lines are generally short already, I did a test and saw minimal savings for them, so decided not to implement delta coding for the "M" lines for now.

I then implemented binary serialization. As I mentioned, I kept the original imComp.txt for debugging purposes so I can easily see the compression format is working properly, but store the full binary file as imFullComp.bin.

Lastly, I implemented Huffman coding and logged arithmetic coding as future work as we discussed. I incorporated WebP as another benchmarking compression algorithm.

3. v5.0/v5.1 alignment with the methodology:

As I mentioned I made new files for these (v7.2 and v7.3), starting by copying what I previously had and adding in all of the advancements I have made up to v7.1 to these studies. Then I added in LPIPS and MSE metrics for the full image to v7.2, as well as the MSE decomposition and interaction term. For some of my images I noted that the interaction term is quite high which implies error is not solely those three buckets, but for most of the kodak and BSDS500 images the interaction term shows me that our decomposition framework captures the images' error sources well. For images where the interaction term is large, should I interpret this as a limitation of the additive decomposition on those image types, and how would you recommend framing this in the paper? I am thinking we could describe it as an indicator of how informative the decomposition framework is: if the interaction term is small, it is more informative and when it is large, the error sources likely interact more significantly, which could be a useful diagnostic on its own.

4. Matched-budget boundary comparison:

I had some difficulty determining exactly how to get to an apples-to-apples comparison for the v7.3 implementations, in particular for B-Spline/Bezier to Chebyshev. For the B-Spline to Bezier comparison, I matched up their number of anchor points N , which was pretty natural. With Chebyshev, I set the degree to 3, since that is what B-Splines and Bezier both use (cubic), and then matched on total stored scalar values per coordinate. Each Chebyshev segment stores 4 coefficients (degree 3 + 1), so to match a budget of N , I use $N/4$ segments. This way all three methods store $2N$ values total (N per coordinate), making the comparison fair. A caveat is that B-splines also store a knot vector ($N+4$ values for cubic), which is overhead the other two methods don't have. Do you think I should adjust my comparison to account for this or simply note it in the paper as a caveat? Does this comparison seem logical and fair to you? I have to incorporate my original polynomial method as well still, but expect it will look similar to Chebyshev.

For benchmarking the comparison, I originally had the same metrics as v7.2 (full MSE, PSNR, SSIM, LPIPS; boundary and interior MAE, PSNR, and SSIM), but since we are focused on boundary encoding specifically in this study, I think it makes sense to only report the boundary metrics. Additionally, after some research I think including an intersection over union (IoU) comparison between the original traced contour and fitted boundary seems natural as well. Please let me know if you think I am on the right track for this.

Doing some testing, B-Splines are generally best for boundary MAE, PSNR, and SSIM. However, I am seeing Bezier and Chebyshev outperforming B-Splines for IoU, which is contrary to what I expected and does not match visual inspection of the reconstructions. I have to spend more time to ensure I have

properly implemented all of these, but I figure this could make sense due to the cost of smoothing and continuity enforcement.

5. Dataset split and hyperparameter protocol:

I downloaded BSDS500 and kodak datasets (I added the citation for BSDS500 to the paper). I also collected ~40 SVG images from freesvg.org to test with. I have not found any citation for the Kodak or SVG datasets and do not believe that it is required. My plan is to use the BSDS500 dataset for validation and the independent final hyperparameter tuning, and use the Kodak dataset for final figures and numbers, as it seems to be the most widely accepted norm for codecs. Please let me know if you recommend an alternative.

Other updates:

I have adjusted my personal TODO vs future works notes accordingly from your last email and currently hold the only remaining code-specific TODO's as:

- Adding original polynomial boundary encoding to v7.3
- Determining final hyperparameter values based on validation sets
- Reporting final byte counts, figures, and other data (which as you have said will require some tweaks and become more clear as I write up the results section)

Next steps:

- Finish the last few coding TODO's
- Section 4-6 writeup (in parallel with getting final figures/tables and adjusting code as needed)
- Finish updating methodology for last round of feedback

Summary of questions:

- Biggest concern: Feedback on if you think I have adequately matched parameter budgets for v7.3. Thoughts on reporting IoU as a metric for the boundary representation ablation and your interpretation of IoU curve.
- B-Spline knot vector overhead caveat note vs code adjustment
- Your thoughts on how to frame in the paper that the polyline fallback is rarely if ever being triggered
- Large interaction term interpretation
- Usage of datasets for validation/testing
- As always, I would appreciate any new feedback you have on the paper draft. I have made a lot of updates to the code since we last spoke as well and your concurrence that I have met the TODO's we prescribed would be helpful.

Thanks again! Not sure if you are a football fan but looks like England is doing quite well! Unfortunately, the US is done, though I didn't expect them to make it far. My girlfriend is Argentinian so pulling for them and England for my grad school friends now!

Kind regards,
Daniel

From: Chris Town <cpt23@cam.ac.uk>
Sent: Thursday, June 18, 2026 5:33 PM
To: D.M. Fisher <dmf41@cantab.ac.uk>
Subject: Re: Revisiting 2021 Computer Vision Project for Possible Publication

Hi Daniel,

As promised, here is more detailed feedback on your latest paper draft.

Thanks for the update, and for the separation of TODO items versus future work. I have read the redrafted abstract and introduction and the full methodology section. The methodology in particular has already come a long way from your previous draft.

A few comments on each section:

Abstract

I suggest that you further simplify some of the sentences, mainly to make the claims clearer:

- "To quantify region-based artifacting": "Artifacting" is a bit odd. "Artefacts"/"artifacts", "reconstruction artefacts", or simply "distortions" is more standard.

I suggest

"To quantify region-based artifacts, we introduce a three-component error decomposition framework. By isolating specific failure modes, this framework provides a principled method for linking compression algorithm design choices to resulting reconstruction errors."

- "For region-based codecs, this framework points to boundary error dominating reconstruction losses on structured imagery..."

I'd prefer "region-based methods" rather than "codecs", since you are here talking about the image compression literature more broadly rather than specific implementations (like a JPEG codec).

It would be stronger to also replace "points to" (provided the paper shows enough evidence) and again simplify the sentence structure, e.g. "Applying this framework reveals that boundary errors dominate reconstruction losses in structured imagery, whereas appearance errors drive distortions in natural imagery. Crucially, this indicates that segmentation is no longer the limiting factor on the evaluated image classes."

Introduction

Much improved from the previous draft.

I think the main issue now is some repetition between the first two paragraphs. Both list some standard codecs, and both make the point that region-based methods are usually evaluated end-to-end without attributing errors to particular stages. Merge these into one tighter paragraph and use the saved space to state the gap more clearly.

I would also make explicit that the main contribution is the error-decomposition framework, and that the codec is the concrete system used to demonstrate it. The codec itself is interesting, but the decomposition framework is the more general contribution.

In the contributions list itself, the item "A local artifacting-influenced per-region planar-only vs DCT-and-planar encoding decision" is awkward. I would replace it with something like "A per-region mode decision between planar-only and planar-plus-residual reconstruction".

Methodology

The methodology section is now in good shape. A few small technical things to fix:

- There is a notation overload involving "k". You use k as the region index in R_k , θ_k , m_k , etc., but the B-spline equation also uses k for spline degree. I would use p for spline degree, which is common in spline notation, or just say "cubic" in words.
- Check the small-region thresholds. One part says the closure/pre-processing step requires $|B_k| > 6$, while the B-spline fallback paragraph says fitting may fail for $|B_k| < 5$. If those are genuinely different checks, explain the difference. If not, use one consistent threshold.
- The adaptive smoothing text says the result is clipped between $s_{\max}$ and $s_{\min} = 0$. That should be "between $s_{\min}$ and $s_{\max}$ ".
- In the planar-fit section, if you mention regions with too few pixels to determine a plane, remember that the issue is not only having fewer than three pixels, but also whether the points are non-collinear.
- "Because the fit is performed in CIELAB, the resulting planar fit has perceptual meaning" is a little too strong. I would say something like "the residuals are more perceptually meaningful than RGB residuals", which is true without implying full perceptual uniformity.
- DCT residuals: The current rule (store a residual when within-region PSNR under planar-only falls below a target) is a reasonable first implementation, but for the paper it will read more convincingly as an approximate rate-distortion decision rather than a pure quality threshold, i.e. one that accounts for both the reduction in distortion and the byte cost of the residual. I would describe the PSNR threshold as a first-order proxy and note that a fuller rate-allocation strategy is future work.
- The error-decomposition section is the strongest part of the current draft. Defining the terms in MSE and then reporting the interaction term is exactly the right approach. I would just add one sentence making explicit that d_C , d_D , d_F , and d_E are all measured against the original image using the same MSE definition.
- Be slightly careful with "MSE is additive". Squared error is additive over pixels, which is what makes MSE a sensible space in which to define these finite-difference terms, but the attribution itself is still an ablation/finite-difference framework, not a physical decomposition.
- Make sure the small-versus-large interpretation of d_{int} is spelled out in prose as well as in the equations.
- More generally, be careful with anything described as part of the proposed codec/method that is not yet actually fully implemented. The per-region DCT residual, mode flags, binary serialisation, and entropy coding are all described in the methodology but I think are still v7.1 TODOs. You have flagged this already in your notes, but keep track of

it. The file-format subsection currently states that the binary entropy-coded format is used "throughout Sections V to VI for benchmarking", which will only be true once the code for that works.

Draft sections still in note form

It is fine that Related Work, Experimental Design, Results, Discussion, Limitations, and Conclusion are still partly in bullet form. I would not spend time completing the results writeup until the byte-counted codec data exist.

The Related Work section that most needs substance is the region-based/vector-hybrid part. That is where you can highlight a novel contribution: previous region-based methods are usually evaluated with aggregate metrics rather than a component-wise attribution of error. Find a few representative papers that show this. You correctly identify the Balle et al. hyperprior and the strong Rate-Distortion (R-D) performance of autoencoders. Clearly state that your goal is not to beat them on R-D, but to offer the interpretability they lack.

TODO versus Future Work

You have arithmetic coding, fixed-point coefficients, and delta coding all in the minimum-implementation group. The genuine minimum for credible rate-distortion numbers is binary serialisation, byte counts, and a simple entropy coder. Huffman or even well-documented quantised binary serialisation may be enough for a first publishable version, provided you are clear about what is and is not compressed. Arithmetic coding in particular can become tricky, and your central contribution does not depend on winning on rate-distortion, so don't let coefficient-coding optimisation hold up the core experiments.

The per-region DCT residual and mode flag are still important if they remain part of the methodology. If v7.1 slips or the results are not clear cut, the text should be changed to "optional extension" or moved to Future Work rather than presented as a completed evaluated method.

I agree that Suzuki-Abe hierarchy, iterative seed re-estimation, pyramids, higher-order interiors, and more elaborate entropy modelling should stay in Future Work unless any of them are needed to make the results more convincing.

Suggested order of work

Within your stated plan, I would prioritise as follows:

1. The Bugs Bunny S/L ratio check. This is quick and determines what the ablation table can claim.
2. The minimum v7.1 codec features for rate-distortion: mode flags, DCT residuals, simple entropy coding, quantisation, binary serialisation, byte counts.
3. v5.0/v5.1 alignment with the methodology: MSE, LPIPS, same pixel set across conditions, and the interaction term.

4. Matched-budget boundary comparison: B-spline, Bezier, Chebyshev, and the original polynomial method if possible.

5. Dataset split and hyperparameter protocol before final numbers are reported.

Keep going on the methodology and introduction now, but leave the final Results and Discussion sections until the byte counts and ablation numbers are known. You can still add figure/table placeholders now, which will help make the missing experiments more explicit.

You have made excellent progress, and the paper now has a much clearer structure, well done!

Best wishes,
Chris

On 14/06/2026 23:24, D.M. Fisher wrote:

> Hi Chris,

>

> I greatly appreciate your advice on the PhD opportunity. The adjusted
> framing of finding the fastest route to doing the work I desire is very
> helpful. The most ideal scenario I think would be similar to what it
> sounds your path was and to make my PhD my work for the company and get
> paid while doing it.

>

> I've spoken a few times now with a senior scientist at CFS on the team I
> hope to be on, so for the time being (until I get out of the navy in
> summer 2027) I am working to foster that relationship and assess his
> guidance. As you mentioned, it is best to communicate with them to
> ensure I am aligning with the desired outcome. While I am very eager to
> get involved with their research in the short term, this time I have
> before my navy contract is up is also a good opportunity to figure out
> precisely what angle is best to do the work I hope to.

>

> I really appreciate your high praise as well and I'm sure this project
> will help future applications, and it has also considerably sharpened my
> research and technical writing skills, in addition to refresh and expand
> my computer vision background. I recognize this may be a somewhat
> unconventional collaboration, and I am very grateful for your guidance.

>

>

> On to the project:

> I have redrafted portions of the abstract and introduction with a focus
> on the portions you flagged and would appreciate your feedback. I
> recognize that all sections are likely to require several iterations to
> hone in on precise language.

>

> I have a draft of the methodology section now and would appreciate
> additional feedback on that as well. I strove to incorporate the
> recommendations you've included in previous emails, and it is written to
> the minimum I plan to implement, which is slightly ahead of where the
> code currently is. As you advised, I don't want to fall into the trap of
> writing it to my goal and forget to implement something, leading to
> overclaiming, so I have kept organized notes throughout this process on

- > outstanding items. Please see my breakdown below of what is currently in
- > my outstanding TODO's vs Future Works (some of which may shift to TODO's
- > when I get the next elements complete). I would appreciate any
- > adjustments you have for shifting or adding things here too.
- >
- > *TODO (minimum implementation):*
- > Codec completion (v7.1):
- >
- > *
- > Per-region DCT residual with mk flag serialized to file
- > *
- > Local rate-distortion metric for planar-only vs. planar+DCT decision
- > (distinct from segmentation threshold)
- > *
- > Binary serialization of full record structure with actual byte counts
- > *
- > Entropy coding method (arithmetic coding) applied to binary format
- > *
- > Replace floating-point coefficients with fixed-point/scaled integers
- > *
- > Apply delta coding to coefficients before entropy coding
- > *
- > S/L encoding ratio check per image (instrumented in encoding loop)
- >
- > Ablation and evaluation (v5.0/v5.1):
- >
- > *
- > Add LPIPS and MSE to v5.0 ablation notebook
- > *
- > Align v5.0 with v7.1 pipeline (8-connected segmentation, same pixel
- > set across all six conditions)
- > *
- > Bugs Bunny ablation: C vs. F results and polyline fallback percentage
- > *
- > Verify matched parameter budgets for B-spline vs. Bezier comparison
- > (v5.1)
- > *
- > Adaptive smoothing parameter ablation
- > *
- > SigmaColor and threshold interaction grid
- >
- > Experimental protocol:
- >
- > *
- > Acquire and prepare standard datasets (Kodak, BSDS500, structured
- > graphics set)
- > *
- > Define validation/test split; tune all hyperparameters on validation
- > set only
- >
- > *
- > *
- > *Future Work:*
- > Boundary and contour representation:
- >
- > *
- > Thin-stroke pathway: lightweight edge/stroke overlay as third

- > encoding mode for cartoons
- > *
- > Adaptive boundary model: detect high-curvature/corner points, split
- > contour or increase knot multiplicity
- > *
- > Polyline fallback triggered by undercoverage at encoding time
- >
- > Topology:
- >
- > *
- > Replace Painter's algorithm and single-loop Moore-neighbor trace
- > with Suzuki-Abe contour hierarchy (cv2.findContours with RETR_CCOMP/
- > RETR_TREE) to handle holes and multi-loop regions
- >
- > Segmentation:
- >
- > *
- > Iterative region-mean re-estimation during segmentation (current
- > seed-based similarity mask is a known source of threshold sensitivity)
- >
- > Interior modeling:
- >
- > *
- > Higher-order polynomial interiors (quadratic: 18 coefficients/
- > region, cubic: 30, quartic: 45) as alternative to DCT residual
- > *
- > Global rate allocation across regions (current per-region decision
- > is local-only)
- >
- > Scalability:
- >
- > *
- > Multi-scale Gaussian pyramid approach: segment at low resolution,
- > propagate and refine near boundaries at higher resolution
- > *
- > Region-local processing to replace remaining whole-image passes
- > *
- > Parallelism and GPU acceleration for segmentation
- >
- > Generalization:
- >
- > *
- > Evaluate on medical, remote-sensing, and fine-scale low-contrast
- > imagery where segmentation may still be limiting
- > *
- > Contextual entropy modeling to tighten R-D curves
- >
- >
- >
- > My next focus is to refine v5.0 (error decomposition ablation), v5.1
- > (boundary representation comparison), and v7.1 (fully implemented codec)
- > per the TODO's above in parallel with writing sections 4-6, followed by
- > section 2.
- >
- > As always, I would appreciate any course correction or prioritization
- > recommendations you have as well. Thanks again!
- >

> Warm regards,
> Daniel

>

> -----

> *From:* Chris Town <cpt23@cam.ac.uk>
> *Sent:* Saturday, May 30, 2026 6:41 PM
> *To:* D.M. Fisher <dmf41@cantab.ac.uk>
> *Subject:* Re: Revisiting 2021 Computer Vision Project for Possible
> Publication
> Hi Daniel,

>

> I am glad that you are getting closer to defining what sort of research
> career you would like to embark on, you have some fantastic
> opportunities ahead of you! I know only a little about fusion power
> other than listening to a few podcasts and reading popular science
> articles. However, personally I consider fusion power to be essential
> for humanity to realise its next "great leap forward", so the more smart
> people such as yourself work in that field the better!

>

> I think the question is not actually "to PhD, or not to PhD", but rather
> what is the fastest route to doing the specific work you want to do? It
> is a great opportunity to work in an industry that is at an inflection
> point with tremendous potential to positively shape the future, and my
> general advice would be to minimise your "expected future regret". The
> advice you got from CFS, combined with your statement that "I'd rather
> be hands-on than spend four to five years adjacent to it", suggests to
> me that committing to a PhD is perhaps not the best path for you at this
> stage. It seems that you could make meaningful contributions at a
> company such as CFS in the near future whereas starting a PhD might
> delay or complicate that particular opportunity. The fact that many of
> their senior people have PhDs is, as they say, largely a result of
> having spun out of an MIT lab, not evidence that a PhD is a requirement
> for this kind of frontier work.

>

> I am not of course diminishing the value of a PhD. The capacity for
> self-directed research is what distinguishes people who can run major
> technical programmes from people who can execute well within them.
> However, that capacity can also be built inside the right industrial
> research environment (which is what AT&T Labs Research had when I worked
> there, before they massively downsized), and the CFS/DeepMind/MIT
> intersection seems to be that kind of environment. This materially
> weakens the usual "you need a PhD to do real research" argument in your
> specific case.

>

> So if your priority is to be hands-on during the upcoming 2027-2028
> fusion testing window, starting at CFS or a similar company directly is
> the more direct path, and that window of opportunity could close before
> long, whereas you still have many years during which you could
> potentially go back to university to complete a PhD (perhaps with
> industrial sponsorship: I got AT&T to pay for mine while they also paid
> me a salary :-)

>

> It is probably a good idea to be quite open about this with CFS or any
> other potential employer before you commit. Would they potentially hire
> you next year into a role that excites you (i.e. something close to
> disruption prediction, plasma control, diagnostics, or scientific ML
> with strong technical mentorship) without a PhD, and would they

> seriously consider supporting you or sponsoring you to complete a PhD
> (perhaps part-time, working on a collaborative research project) in
> future once you have proven yourself?
>
> This project you and I are doing is actually relevant evidence to inform
> your own decision. You have shown over these past months that you can
> take an interesting open-ended problem, restructure it, drive it forward
> while responding very well to guidance, learn what you need as you go,
> and push it toward a publishable result. That is precisely the core
> skill a PhD is meant to build. You are demonstrably capable of it
> already, which tells me you do not need a PhD to prove you can do
> research, which in turn means, if you do one, it should be because you
> want the depth and the dedicated time to think deeply about a particular
> problem domain, not because you need the certification. And getting this
> paper published will itself strengthen any PhD application considerably,
> as well as your standing for a research-facing industrial role, so it
> serves both paths :-)
>
> Speaking of which, thanks also for sharing your draft paper writeup! I
> will take a peek from time to time, but for me it would be easiest if
> you sent me an email (like you have been doing) whenever you feel that
> you'd like my input, or simply to share another substantive milestone.
>
> Please keep progress on your code (v7.1 and beyond) moving in parallel,
> but do not let it block the paper. The algorithmic work has reached the
> point where writing will clarify what still needs to be implemented,
> measured, or left as future work.
>
> If methods and ideas such as thin-stroke handling and per-region DCT
> mode selection work well, they can be moved into the main method and
> results. If they do not mature in time, they can stay as future work.
> All worthwhile research is inherently work in progress, and so any paper
> can only present a snapshot (with results and data analysis to ensure it
> is credible) that opens up avenues for future work.
>
> Overall, the structure of your draft is already good. The main framing
> is also much stronger than where we started. Here are a few high-level
> comments on the draft:
>
> - Be careful not to overclaim in the abstract and introduction. For
> example, phrases like "region-based image compression methods currently
> lack a robust framework" are too broad. It would be safer to say that
> existing region-based methods are often evaluated end-to-end, making it
> difficult to attribute reconstruction error to segmentation, boundary
> approximation, or appearance modelling separately.
>
> - The human-vision motivation is interesting but too general and a
> little imprecise physiologically (human visual perception is really
> driven more by the LGN and cortex). A more direct connection to your
> methods would be to point out that standard raster codecs operate on
> pixels or blocks, while many images contain coherent regions whose
> boundaries and interiors can be modelled explicitly.
>
> - The paper should highlight the error-decomposition framework as a
> central contribution, with the codec as the concrete system used to
> demonstrate it. The paper can still include rate-distortion curves, but
> it is best not to make a comparison of raw compression performance

> against JPEG/WebP/learned codecs the only standard by which the work is
 > judged. Other region-based codecs usually report a single aggregate
 > distortion number, so they cannot tell you whether a given
 > reconstruction error came from segmentation, boundary approximation, or
 > interior modelling. In contrast, you provide a decomposition that
 > separates these, and that separation in turn motivates a per-region
 > encoding decision. Your error-decomposition section is already one of
 > the strongest parts of the paper, and I suggest you expand on this.
 >
 > - Use MSE or another additive distortion measure when defining the
 > decomposed error terms, because PSNR itself is not additive. You can
 > still report PSNR, SSIM, and LPIPS for interpretability, but the
 > decomposition should be grounded in MSE or summed per-pixel squared
 > error. However, even with MSE, I would describe the A-F conditions as an
 > ablation or finite-difference attribution framework, not as a perfectly
 > additive physical decomposition. Boundary and appearance errors can
 > interact. A defensible approach is to define the main terms using MSE:
 > $d_{\text{seg}} = dC$
 > $d_{\text{bnd}} = dF - dC$
 > $d_{\text{app}} = dD - dC$
 > where C is exact contour + original pixels, F is spline contour +
 > original pixels, and D is exact contour + planar appearance. The
 > predicted error of the full no-residual method E is then:
 > $d_{\text{pred}} = d_{\text{seg}} + d_{\text{bnd}} + d_{\text{app}} = dD + dF - dC$
 > Then explicitly report the interaction term:
 > $\text{interaction} = dE - d_{\text{pred}} = dE - (dD + dF - dC)$
 > If this interaction is small, it strengthens the claim that the three
 > buckets explain most of the error. If it is not small, that is still
 > useful, because it shows that boundary and appearance errors are not
 > independent.
 > So the plan is to
 > (a) define the decomposition in terms of MSE or summed per-pixel squared
 > error, where adding and subtracting error terms is meaningful;
 > (b) report how closely $d_{\text{seg}} + d_{\text{bnd}} + d_{\text{app}}$ predicts dE , ideally with the
 > interaction term as a small table or figure;
 > (c) then report PSNR, SSIM, and LPIPS as reader-facing summary metrics.
 > Also make sure all conditions are evaluated on the same pixel set, or
 > with a clearly fixed boundary/interior mask, so that differences between
 > conditions reflect the intended error source rather than a changing
 > evaluation region.
 >
 > - The "resolution-independent" claim is now mostly handled correctly,
 > but keep it precise throughout. The boundary plus planar base layer is
 > vector-like and can be evaluated at arbitrary resolution. The DCT
 > residual, however, is raster-bound and tied to an encoding resolution.
 > So the safest claim is that the base layer is resolution-independent,
 > while the residual stream improves fidelity at a chosen raster resolution.
 >
 > - The current writeup mentions optional per-region DCT residuals, mode
 > flags, binary entropy-coded format, and some v7.1 ideas. Be careful that
 > any methods that are not yet fully implemented are flagged up as "to do"
 > items somewhere.
 >
 > - The related-work section is currently a set of relevant headings and
 > notes. I would make sure it covers classical codecs, learned
 > compression, region-based/second-generation image coding,
 > raster-to-vector conversion, spline/contour representations,

> segmentation/superpixels, and residual transform coding. You do not need
> to beat learned codecs on raw rate-distortion performance, but you
> should acknowledge them and position the contribution of your work as
> interpretable and diagnostic rather than purely performance-driven.
>
> - The experimental design section is shaping up well. Keep the
> validation/test distinction explicit. Hyperparameters such as
> sigmaColor, threshold endpoints, spline smoothing constants, residual
> thresholds, and polyline stride should be chosen on validation images,
> not on the final test images. This will make the results much easier to
> defend.
>
> - Add placeholders for all figures/tables, even if the final data is not
> ready yet. Write the experimental protocol before running final
> experiments, especially dataset split, metrics, baselines, and
> hyperparameter tuning procedure. Then run the experiments needed to fill
> the results section.
>
>
> You have made an excellent start on the paper and have plenty of
> material to flesh most of it out already. I would keep going on the
> method and introduction sections now and leave the results prose until
> the byte-count numbers are settled.
>
> Excellent work, as always!
>
> Best wishes,
> Chris
>
> On 27/05/2026 13:00, D.M. Fisher wrote:
> > Hi Chris,
> >
> > Glad to hear Cambridge is warming up! I have so many great memories
> > enjoying the greens with friends during my time there. Thank you again
> > for the most recent round of feedback, I have taken note of all of them
> > on the document I've maintained to track feedback and TODO's. Thank you
> > for the advice on the literature search as well, I will let you know if
> > I have difficulty as I move through drafting my Related Works section. I
> > also appreciate the clarification on how to frame this collaboration on
> > my CV, and have adapted the layout you suggested.
> >
> > I had mentioned back when I first reached out to you that I was
> > considering a PhD program at the intersection of nuclear fusion and CS
> > and have since made a great deal of progress in parallel with this
> > project on figuring out what I want to study and narrowing down my
> > specific area of interest.
> >
> > A significant challenge the magnetic confinement fusion companies are
> > facing is regarding the prediction and mitigation of plasma disruptions,
> > and they are approaching the problem with some very interesting physics-
> > based and data-driven solutions. There seem to be a lot of really
> > promising open areas of research at this junction, and I have been in
> > early conversations with members at Commonwealth Fusion Systems (CFS),
> > and their collaborators at Google DeepMind and MIT to learn about their
> > efforts and assess how to best position myself to make a meaningful
> > contribution as I transition out of the Navy in August 2027.
> >

> > The open question I'm wrestling with is whether to go directly to CFS in
> > 2027 or do a PhD first. In the short-term, I want to make a positive
> > contribution to the efforts leading to the initial success of the
> > technology; my longer-term aim is to rise through a leading fusion
> > company (I am primarily focused on CFS) and eventually help lead the
> > global rollout of fusion energy. The case for going straight in is that
> > the work is happening now (CFS expects to begin testing in 2027–2028)
> > and I'd rather be hands-on than spend four to five years adjacent to it.
> > The case for a PhD is that I do really love research generally, the
> > technical depth seems to compound, and most of the people leading these
> > efforts have one. The researchers at MIT and GDM seem to co-author with
> > the CFS team on their research as well, so they still seem to be deeply
> > involved. However, when I've asked leadership at CFS for their thoughts,
> > they have been advising me that it isn't necessary and the people who
> > have PhDs simply happen to, given that the company came out of MIT's
> > research lab. I'd value your thoughts on the broader question of when a
> > PhD is the right path to the kind of work one wants to do versus when
> > it's worth going directly to the work itself. I'm sure you've seen both
> > routes play out many times in your career.

> >

> > On the paper:

> > I've translated all of our notes on publication framing, as well as
> > other notes I've been taking throughout the process, into a LaTeX
> > skeleton structured around the section layout we discussed, and I have
> > begun filling in sections and elaborating on notes to translate them
> > into prose. I've shared the Overleaf file with you so you can follow
> > along with my drafting progress, add comments directly as desired, or
> > make any edits you see fit. Please let me know if there is a more
> > effective or otherwise preferred way to share this.

> >

> > So far I've drafted the abstract and introduction, and am now working
> > through the methodology section. The remaining sections are scaffolded
> > as bullet-point notes that I'll expand into proper paragraph form as I
> > work through them. For now, I've listed several of our outstanding ideas
> > as future works until I have the bulk of the paper complete, and plan to
> > transition ideas from that section into the body of the methodology as
> > we see fit pending additional features added.

> >

> > I'm happy to take comments at any time in Overleaf or via email, and
> > still intend to send you updates via email periodically of major
> > milestones in drafting and the algorithm development.

> >

> > Kind regards,

> > Daniel Fisher

> >

> > -----Original Message-----

> > From: Chris Town <cpt23@cam.ac.uk>

> > Sent: Friday, May 1, 2026 19:27

> > To: D.M. Fisher <dmf41@cantab.ac.uk>

> > Subject: Re: Revisiting 2021 Computer Vision Project for Possible
> > Publication

> >

> > Hi Daniel,

> >

> > Cambridge is finally experiencing spring sunshine, just in time for the
> > undergraduates having to revise for their exams :-)

> >

> > I am pleased that you have made substantial progress again, including
> > almost all of the high-priority items I suggested before. Several of the
> > changes (especially the move to standard L-a-b distances and the
> > adaptive smoothing for splprep) seem to have given exactly the kinds of
> > improvements I was hoping for, well done! The grid search results are
> > also a nice piece of analysis in their own right, and the pipeline is
> > now much closer to a paper-ready baseline.
> >
> > Here are my thoughts on your latest updates and questions:
> >
> >
> > Grid search results
> >
> > Something interesting jumps out from your sigmaColor x thresh table that
> > I would explore a bit further before treating "sigmaColor=35" as
> > optimal. At every sigmaColor row, going from thresh=4 up to thresh=12,
> > the PSNR actually increases (e.g. at sigmaColor=35, 20.6 -> 22.1 dB),
> > and file size and region count fall mostly as expected (although not
> > perfectly monotonically). Naively one might expect the opposite: a
> > looser threshold means more aggressive merging, which should hurt PSNR.
> > The fact that it does not is consistent with the notion that very low
> > thresholds produce a flood of tiny regions that the spline+planar model
> > cannot represent efficiently or accurately. Each tiny region contributes
> > its own small contour error and spline overshoot, and those errors
> > accumulate across hundreds or thousands of regions. Slightly more
> > aggressive merging gives you fewer but larger regions that the planar
> > model fits better.
> >
> > If that interpretation is right, there is a "sweet spot" beyond which
> > more aggressive merging finally does start to hurt performance, but it
> > is just not at the values you tested. I would extend the sweep upwards
> > (thresh=14, 16, 18, 20) to find where PSNR starts to fall again. I would
> > also repeat this on a second representative image of a different
> > category (cartoon vs natural photo, also try higher resolution images)
> > before fixing sigmaColor=35 globally, since the optimal balance may
> > shift between content types. The U-shape, once confirmed, is itself
> > worth reporting in the paper because it is interesting in that it goes
> > against a (to me) intuitive reading of the threshold parameter.
> >
> > A more general note on hyperparameter discipline: when you fix
> > sigmaColor=35 (or any other parameter) for the final paper experiments,
> > please make sure those choices are tuned on a separate set of validation
> > images and not on the same images you will report final results on.
> > Reviewers are sensitive to apparent overfitting when a method has
> > several adaptive rules, so being explicit in the paper that final test
> > images were never used for tuning is worth doing.
> >
> > I agree that it is best to not go overboard on adaptive hyperparameters
> > more generally.
> >
> >
> > Code review of v7.0
> >
> > I have briefly looked through the v7.0 notebook. Most of the changes are
> > solid and the pipeline is noticeably tighter than in v6.0, but there are
> > some things I would like to mention.
> >

> > Contour tracing and holes: "traceClusterEdges" still does a single
> > Moore-style trace starting from one seed pixel. That is fine for
> > simply-connected regions but it cannot capture inner contours (holes) or
> > multiple boundary loops on its own. If a region has a hole, only one of
> > the two boundary loops will be traced. The border_mask in segmentation
> > includes both outer and inner boundary pixels, but the trace will pick
> > only one. This might be contributing to topology issues on cartoon
> > images where one region wraps around another. The fix is non-trivial:
> > you would either need to do connected-component labelling on the
> > border_mask itself to find each separate boundary loop and trace each,
> > or move to a contour retrieval algorithm like Suzuki-Abe
> > (cv2.findContours with RETR_CCOMP or RETR_TREE) which gives parent-child
> > hierarchies directly. cv2.findContours would also remove most of the
> > custom tracing code. Don't spend too much time on this, but it is worth
> > doing once the paper structure is in place, especially because explicit
> > hole handling may help with the "Bugs Bunny" failure modes.
> >
> > Spline rounding clamping: in the decompression cell, the L (polyline)
> > fallback path clamps points to the image bounds with np.clip, but the S
> > (spline) path does not appear to do the same. After splev returns float
> > coordinates and you round to int, occasional samples can land outside
> > the image bounds, which then either trigger silent fillPoly clipping or
> > produce small artefacts at the image edge. Adding the same clip(0, H-1)
> > / clip(0, W-1) logic to the S path would be a simple solution.
> >
> > Encoding loop print: the line
> > print(f"s={s_val:.2f}, perim=...")
> > inside the encoding loop will produce a wall of output for any
> > non-trivial image. Either remove it or guard it behind a "verbose" flag.
> >
> > Coverage repair fallback: I am glad you added the distance_transform_edt
> > nearest-painted-pixel fallback. The fact that you were finding many
> > regions still incomplete after the dilation + convolution pass is itself
> > diagnostic. It suggests there are cases where the rasterised spline
> > polygon does not cover all pixels labelled as belonging to that cluster
> > by the segmentation. That is consistent with the contour tracing and
> > rounding issues above and would be worth tracking as a metric on each
> > test image ("coverage before repair / after dilation pass / after
> > nearest-painted fallback"). This would be useful both as a sanity check
> > and as a candidate diagnostic figure for the paper. Ideally the core
> > representation should fill almost everything correctly with the fallback
> > used only as a last-resort safeguard. Then if the fallback ends up doing
> > a lot of work it suggests a topology or rasterisation problem.
> >
> > Adaptive splprep smoothing: the contour_complexity_metrics +
> > adaptive_spline_smooth combination looks well thought through. The "0.6
> > * mean_turn + 0.4 * corner_density" weighting and the perimeter-scaled
> > max_smooth are reasonable choices, but they are also somewhat arbitrary
> > and a reviewer may ask why those constants were chosen. I would either
> > run a small ablation showing that the result is not very sensitive to
> > those weights, or position them more cautiously in the text as defaults
> > that worked well empirically. I would also record more than just
> > PSNR/SSIM when reporting the effect of adaptive smoothing. In
> > particular, measure the average number of stored control points and
> > average byte cost per region, so you can show that adaptive smoothing
> > improves the rate-distortion trade-off and not just quality.
> >

> > Reproducibility: the repository currently uses an absolute Windows path
> > for image loading and selects images by list index (chosenImage =
> > pics[9]). Before you start running the experiments that will go into the
> > paper, I would move to relative paths, explicit filenames, and a small
> > driver script or config block that makes the experimental protocol
> > obvious from reading the code.
> >
> > README: It is slightly out of date. Ensure you update the descriptions
> > to clearly distinguish what is done in v7.0 from what is currently in
> > progress for v7.1.
> >
> >
> > Publication framing document
> >
> > This is well organised and it tracks accurately what we have discussed.
> > I would be cautious with the following two phrases though:
> >
> > "Resolution-independent" needs qualification. The boundary spline plus
> > planar interior model is genuinely vector-like and can be evaluated at
> > any resolution. The DCT residual stream, however, is block-based and
> > tied to a specific raster resolution. So the accurate claim is something
> > like: "the base layer is vector-like and resolution-independent in its
> > boundary and region representation, while the optional residual stream
> > improves texture fidelity at a chosen raster resolution." That is still
> > a real advantage worth highlighting, but it just needs to be stated
> > precisely so a reviewer cannot object to the stronger claim.
> >
> > "Semantic image compression" is also worth rethinking. In the current
> > literature, "semantic" tends to imply object-level or category-level
> > understanding (typically from a learned segmentation network). Your
> > method is region-based and structurally adaptive, but it does not
> > understand semantic categories in that sense. I would be wary of using a
> > term that invites a comparison your method cannot fully deliver on.
> > Alternatives that are more accurate might be "region-based vector-hybrid
> > image representation", "segmentation-driven hybrid compression", or
> > "structure-aware image compression". You can also still describe the
> > work as content-adaptive, which is definitely true.
> >
> > Under "Key Novel Contributions", I would add the standard L-a-b delta-E
> > threshold as its own item. That switch was not just a mere code change,
> > it gave you a perceptually interpretable threshold parameter, which is
> > itself a small methodological contribution and one that reviewers should
> > appreciate when reading the segmentation section.
> >
> > Under "Evaluation", the bullet point on "rate-distortion curves (bits
> > per pixel vs PSNR)" is the right anchor metric, but I would also add
> > SSIM and LPIPS (Learned Perceptual Image Patch Similarity) as secondary
> > perceptual metrics. PSNR alone can undervalue perceptual quality on
> > region-based methods, and structured/cartoon imagery is precisely where
> > SSIM tends to disagree with PSNR. Including LPIPS, even just on a subset
> > of images, will pre-empt a likely reviewer comment.
> >
> > Under "Ablation & Analysis Opportunities", I suggest you add the
> > sigmaColor x thresh interaction grid (you have already started this) and
> > the per-image-category breakdown of the three-bucket error
> > decomposition. The latter is one of your strongest evaluation
> > contributions and it deserves explicit mention in the abstract and

> > introduction, not just in the results section.
> >
> > Under "Limitations & Future Work", add the topology / hole-handling
> > point flagged above.
> >
> > Under "Related Work & Context", I would add comparison against (or at
> > least acknowledgement of) deep-learned image compression methods, e.g.
> > Balle et al.'s end-to-end optimised hyperprior/autoencoder codecs, even
> > just to position your work as a deliberately interpretable,
> > hand-engineered counterpoint. These are now the de facto baseline that
> > any new compression method gets compared against, and a reviewer may ask
> > about them. You do not need to beat these methods (your contribution is
> > structural and interpretive, not raw R-D performance), but you do need
> > to acknowledge them.
> >
> > When v7.1 is complete, I would also recommend that the file format make
> > per-region encoding decisions explicit. Maybe a mode flag per region
> > distinguishing planar-only, planar plus DCT residual, and (if the
> > thin-stroke pathway works) stroke-overlay. The decision rule itself is
> > something the paper can describe and ablate, but only if the format
> > records it.
> >
> >
> > Open questions and remaining list
> >
> > Your enumeration of remaining items is sensible. I would prioritise them
> > as follows:
> >
> > 1. The Bugs Bunny C vs F discrepancy is the most important one to
> > resolve before writing up the ablation table, and it should be a quick
> > check rather than a big investment. If many regions are using the L
> > fallback rather than spline path, the ablation table in the paper needs
> > to be honest about that. Easiest check: instrument the encoding loop to
> > count S vs L lines per image and report the ratio. That data point is
> > also worth including in the paper.
> >
> > 2. Binary encoding and actual byte-count measurement is the single most
> > important thing for the experimental story. Without it, every
> > rate-distortion claim is provisional. I would push this up the queue
> > once paper drafting begins, because the moment you have a draft method
> > section you will want to populate the results section with credible
> numbers.
> >
> > 3. The boundary-model parameter-budget normalisation for the
> > curve-comparison table is worth doing, but does not need to be perfect.
> > Even reporting "B-spline used N1 control points, Bezier used N2,
> > Chebyshev used N3" alongside the PSNR numbers is enough to give
> > reviewers what they need to interpret the comparison fairly.
> >
> > 4. The thin-stroke pathway and the per-region DCT decision in v7.1 are
> > great additions but are exploratory enough that I would treat them as
> > desirable rather than essential. If they work well in time, fold them in
> > with a clean ablation. If not, describe them in the future-work section.
> >
> > 5. PyrLevels / Mahalanobis / iterative seed re-estimation: unless you
> > feel strongly about any of them, keep these as future work in the paper.
> > You have already invested enough in adaptive hyperparameters and there

> > is a real risk of making the pipeline too configurable and too complex
> > to explain and evaluate cleanly.
> >
> >
> > Literature access
> >
> > The paywall problem for non-affiliated researchers is genuinely awkward
> > but quite tractable. A few practical routes, in roughly the order I
> > would try them:
> >
> > - arXiv (arxiv.org) for almost anything in computer vision from the last
> > decade. The cs.CV section in particular is comprehensive.
> > - Google Scholar, especially the "All N versions" link which often
> > surfaces a free preprint or institutional copy alongside the paywalled
> > version.
> > - Unpaywall (unpaywall.org): I have not used this, but it has become
> > quite popular. It is basically a free browser extension that
> > automatically detects when an open-access version of a paywalled paper
> > exists and links to it. Particularly useful for older papers.
> > - Semantic Scholar (semanticscholar.org) and OpenAlex (openalex.org) for
> > discovery and citation chasing. Semantic Scholar's "References" and
> > "Citing Papers" graph is genuinely very good.
> > - CORE (core.ac.uk) and DOAJ (doaj.org) for open-access aggregation.
> > - PapersWithCode (paperswithcode.com) for any topic with associated
> > implementations - useful for related-work-with-baselines.
> > - Author homepages and group/lab pages - searching "[author surname]
> > [paper title] pdf" usually finds them.
> > - Direct emails to authors. A polite one-line request almost always
> > works. Researchers are generally pleased that someone is reading (and
> > wants to cite!) their work.
> >
> > For this paper specifically, you will probably find that the great
> > majority of what you need is reachable through arXiv, Google Scholar, or
> > Unpaywall. If you have problems with any papers that look interesting,
> > send me the citation and I will try to download them for you.
> >
> >
> > On the CV / role question
> >
> > I appreciate you asking this directly rather than just claiming
> > something. The honest description is that you are doing independent
> > research that I am supervising/advising on a continuing basis, and that
> > this work arose from your MPhil research at Cambridge.
> > I would not use a formal title such as "Visiting Researcher" or
> > "Visiting Engineer" unless there is an actual formal appointment. Those
> > titles usually require formal Departmental or University approval.
> > However, it is factually true that you are an alumnus of the Department
> > and University, as well as your College.
> >
> > On your CV, you could put something like:
> >
> > Independent Research Project, supervised by Dr Chris Town, Department of
> > Computer Science and Technology
> > Image Compression via Colour Segmentation and Parametric Boundary
> > Encoding
> > 2024 - present
> >

> > You are now in a strong position to begin drafting the paper. The most
> > important thing for the next few weeks is to work on the priority items,
> > and increasingly be guided by the writing process itself (as you write
> > the paper, it will become clearer what experiments and improvements are
> > still needed).
> >
> > Excellent work and keep going!
> >
> > Best regards,
> > Chris
> >
> >
> > On 26/04/2026 17:57, D.M. Fisher wrote:
> > > Hi Chris,
> > >
> > > Happy belated Easter to you too! I hope you've been well.
> > >
> > > I really appreciate all of the feedback again in this round and the
> > direction you are providing steering me towards wrapping up the
> > algorithmic work, getting the rigorous data we need, and getting into
> > drafting the paper. I wanted to send a quick update on my progress as I
> > continue moving through this. I've been working through v7.0 now and
> > focused primarily on implementing the changes you highlighted in your
> > review and in my outstanding list.
> > >
> > > The algorithmic changes I've implemented are:
> > >
> > > - Fixed repair_coverage function to resolve early return bug
> > >
> > > - Updated spline decoding to use adaptive perimeter-based sampling
> > scaled by estimated contour arc length and control-point count
> > >
> > > - Updated Painter's ordering to sort decoded regions by true polygon
> > area with cv2.contourArea, instead of contour point count
> > >
> > > - Replaced fixed-stride contour down-sampling in spline fitting with
> > full-contour fitting and `splprep` smoothing (`s > 0`). I thought it
> > would be interesting to make this an adaptive hyperparameter, so I
> > extended this to derive `splprep`'s s per boundary from geometric
> > complexity (perimeter, turn-angle statistics, corner density), and used
> > lower smoothing for sharp/corner-rich contours and higher smoothing for
> > smoother regions. This definitely showed improvements as you predicted.
> > >
> > > - Updated segmentation similarity to use Euclidean distance in CIELAB
> > space, replacing normalization with a ΔE -interpretable threshold. This
> > showed massive improvement!
> > >
> > > - Unified region connectivity to 8-neighborhood across segmentation
> > and contour extraction (CCL, border detection, tracing).
> > >
> > > - Removed redundant mean-color storage, only storing planar M
> > coefficients for reconstruction now.
> > >
> > > - Replaced nested-loop coverage repair with a fully vectorized method
> > using scipy.ndimage.binary_dilation and convolution. I added nearest-
> > painted-pixel fallback with distance_transform_edt because I was finding
> > lots of regions not being fully filled still.

> > >
> > > - Implemented adaptive segmentation threshold based on image-level
> > complexity with it tailored so higher complexity --> lower thresh and
> > lower complexity --> higher thresh.
> > > Kept global Euclidean distance for stability. I'm still tracking
> > local Mahalanobis distance as future work though either for me or to
> > acknowledge in the paper.
> > > Ran grid search over sigmaColor and thresh to analyze interaction. I
> > confirmed the coupling you expected between smoothing and clustering
> > behavior and selected sigmaColor = 35 for now as the best balance of
> > runtime and performance. I considered making this process adaptive too,
> > but then felt maybe I'm going overboard on the adaptive
> > hyperparameters... I included some results for an image below (I tested
> > on other images as well of course but here is an example):
> > >
> > > Results summary:
> > > `sigmaColor=20`
> > > thresh=4 | PSNR=20.092, SSIM=0.9105, size=184849 B,
> > > time=19.86s, regions=2636
> > > thresh=6 | PSNR=20.520, SSIM=0.9177, size=182891 B,
> > > time=10.12s, regions=2272
> > > thresh=8 | PSNR=20.828, SSIM=0.9252, size=183955 B,
> > > time=9.05s, regions=2004
> > > thresh=10 | PSNR=21.708, SSIM=0.9150, size=150181 B,
> > > time=6.94s, regions=1513
> > > thresh=12 | PSNR=22.028, SSIM=0.9170, size=151817 B,
> > > time=6.60s, regions=1350
> > > `sigmaColor=35`
> > > thresh=4 | PSNR=20.564, SSIM=0.9170, size=186603 B,
> > > time=11.37s, regions=2400
> > > thresh=6 | PSNR=21.224, SSIM=0.9006, size=146756 B,
> > > time=7.89s, regions=1792
> > > thresh=8 | PSNR=21.577, SSIM=0.9128, size=146485 B,
> > > time=7.46s, regions=1590
> > > thresh=10 | PSNR=21.939, SSIM=0.9168, size=149306 B,
> > > time=6.82s, regions=1429
> > > thresh=12 | PSNR=22.129, SSIM=0.9185, size=148401 B,
> > > time=6.18s, regions=1299
> > > `sigmaColor=50`
> > > thresh=4 | PSNR=21.164, SSIM=0.9220, size=185375 B,
> > > time=9.92s, regions=2141
> > > thresh=6 | PSNR=21.393, SSIM=0.9135, size=178509 B,
> > > time=8.02s, regions=1770
> > > thresh=8 | PSNR=21.732, SSIM=0.9084, size=150212 B,
> > > time=9.88s, regions=1494
> > > thresh=10 | PSNR=22.004, SSIM=0.9127, size=153880 B,
> > > time=9.52s, regions=1370
> > > thresh=12 | PSNR=22.374, SSIM=0.9202, size=150036 B,
> > > time=7.78s, regions=1204
> > > `sigmaColor=65`
> > > thresh=4 | PSNR=21.281, SSIM=0.9198, size=193251 B,
> > > time=10.61s, regions=2022
> > > thresh=6 | PSNR=21.802, SSIM=0.9229, size=186462 B,
> > > time=8.06s, regions=1695
> > > thresh=8 | PSNR=22.025, SSIM=0.9101, size=154819 B,
> > > time=8.45s, regions=1390
> > > thresh=10 | PSNR=22.704, SSIM=0.9137, size=150340 B,

> > time=8.20s, regions=1168
> > > thresh=12 | PSNR=22.765, SSIM=0.9224, size=149781 B,
> > time=7.57s, regions=1089
> > >
> > > I recently made v7.1 as well, which I am separating from the other
> > updates, as it fundamentally will change the imComp.txt encoding schema.
> > In this script, I intend to implement the "thin dark stroke" and local
> > region-based vs DCT encoding. This portion I intend to keep working on
> > in parallel with the paper.
> > >
> > > Other outstanding items from your last email that I'm still tracking
> > (besides what I mentioned in v7.1) include:
> > >
> > > - For paper results: investigating the Bugs Bunny C vs F result
> > discrepancy from ablation testing and verifying the parameter budget was
> > consistent for B-Spline vs Bezier comparison
> > > - Moving to binary encoding and to measure actual byte size for rate-
> > distortion comparisons
> > > - Exploring a more principled adaptive boundary model for high-
> > curvature regions
> > > - Further examining clustering seed behavior
> > > - Considering the pyrLevels idea
> > >
> > > Going back to your recommended steps to completion, I believe step 1
> > (fixing immediate code issues) and step 3 (experiment with smooth > 0)
> > to be complete with this round of changes. I am now moving into step 2
> > where I intend to begin with my original paper from the MPhil as a
> > starting point and modify the layout to be in line with your recommended
> > sections for the paper. Then I will work through writing up the
> > algorithm to catch up to our current progress. At that point, I will
> > continue with the changes that modify the imComp.txt schema in v7.1
> > (step 4 + thin-dark stroke) and some of the remaining ideas we have,
> > which we can reassess at that point if we should implement them or
> > acknowledge them as future work.
> > >
> > > As I've moved through all of these changes and several rounds of
> > feedback from you, I've been maintaining notes and binning your
> > recommendations into algorithm improvements vs publication framing/
> > future works. As I now move towards drafting the paper, I wanted to
> > share my notes with you and ask if there is anything you'd like to
> > clarify or add on to this, and I have attached the document where I
> > track it to this email. Of course, as you mentioned it will become more
> > clear to us as we make progress on the write up what else may be missing
> > too.
> > >
> > > Since I'm not currently affiliated with a university, I don't have
> > access to a number of papers behind paywalls. Do you have any
> > recommendations on how best to approach this for doing my literature
> > search for the related works section? I previously tried using my alumni
> > email to gain access and unfortunately most of the sites do not
> > accept it.
> > >
> > > My interest in this research is rooted in extending what I think is
> > an interesting idea, learning more about computer vision, improving my
> > research skills as I still consider a PhD for my next stage after the
> > navy, and improving my CV as I look to apply for programs and/or jobs
> > soon. Over the last several months of working on this together, we have

> > very much been achieving these goals from my perspective, and I am very
> > grateful for your collaboration and advising.
> > >
> > > As I continue working on this research, I've been thinking about how
> > to appropriately describe this work on my CV. I want to be accurate and
> > not overstate anything, but also reflect that I've been working on this
> > in a research capacity under your guidance. I know a lot of universities
> > have "visiting engineer" or "visiting researcher" roles within their
> > departments and was curious how you would recommend framing this? If
> > there are any roles you think would be appropriate that require formal
> > application or documentation, I would be happy to do what is needed.
> > >
> > > Thanks again for all of your guidance and I look forward to digging
> > into the drafting process!
> > >
> > > Kind regards,
> > > Daniel
> > >
> > >
> > > -----Original Message-----
> > > From: Chris Town <cpt23@cam.ac.uk>
> > > Sent: Sunday, April 5, 2026 8:10
> > > To: D.M. Fisher <dmf41@cantab.ac.uk>
> > > Subject: Re: Revisiting 2021 Computer Vision Project for Possible
> > Publication
> > >
> > > Hi Daniel,
> > >
> > > Happy Easter!
> > >
> > > Really excellent work on this round of updates. That is a substantial
> > > amount of progress and it shows real momentum!
> > >
> > > I fully agree with your interpretation of the ablation results.
> > >
> > > Condition C (exact contours + original pixel colours) gives near-
> > perfect
> > > reconstruction across all three test images. That is a hard
> > problem and
> > > for these test images you seem to be handling it to a good standard. I
> > > would not spend more time trying to improve segmentation accuracy
> > right
> > > now, though I would be cautious about generalising too far: the claim
> > > the paper should make is that segmentation is no longer the dominant
> > > bottleneck on many image types, not that it is universally solved. Its
> > > importance may still vary, especially for images with very thin or
> > > low-contrast structures that have not been tested yet.
> > >
> > > The boundary error story (A vs B) is particularly clear. For the
> > > penguin, full-image PSNR drops from 25.4 to 22.2 when switching from
> > > exact contours to B-splines. For Bugs Bunny, it drops from 32.0 to
> > 14.7,
> > > and almost all of that damage is in the boundary band (28.2 down
> > to 9.4)
> > > while the interior is essentially unchanged (35.8 to 35.7). That
> > cleanly
> > > confirms the thin-line / sharp-corner vulnerability of smooth B-

> splines.
> > > This is exactly the kind of quantitative finding that a paper should
> > > feature prominently.
> > >
> > > One thing I would double-check in the Bugs Bunny data before
> writing it
> > > up: conditions C and F give nearly identical full-image PSNR (about
> > > 38.5) and identical interior numbers. That is surprising given how
> badly
> > > the spline+mean version (B) degrades. If the spline boundary were
> truly
> > > devastating the contours, condition F should also suffer
> noticeably. One
> > > possibility is that many of the Bugs Bunny regions are falling back to
> > > the polyline (L) geometry rather than the spline path, or that the
> > > relevant decode path is reproducing the masks more faithfully than
> > > expected. It would be worth checking how many regions actually use
> the S
> > > path versus the L fallback. This does not invalidate the overall
> > > conclusion, but it does mean you should understand what is happening
> > > before citing the numbers in the paper.
> > >
> > > For the mountain, your planar fits improve interior PSNR from 24.0 to
> > > 27.4 relative to mean colour, a meaningful gain that validates the
> > > upgrade. But the gap to condition C (40.3) shows that even a planar
> > > model leaves a lot of texture unrecovered, which is exactly the
> > > motivation for the hybrid DCT approach you are already exploring. The
> > > fact that you can point to specific numbers here makes the case for
> > > hybrid encoding much more compelling than a qualitative argument
> alone.
> > >
> > > Condition F (spline + original colours) was a good addition. For the
> > > penguin, F gives 29.1, i.e. substantially better than B at 22.2. That
> > > tells you concretely how much quality is available to recover just by
> > > improving the boundary model, even without touching the interior, and
> > > that is a useful upper bound to explore.
> > >
> > > Overall, you now have a quantitative decomposition of your error
> sources
> > > that very few image compression papers bother to provide. This is a
> > > genuine strength of your work and I would position it as a
> > > methodological contribution in the paper, not just an internal
> > > diagnostic. The three-bucket framework (segmentation error, boundary
> > > approximation error, appearance model error) combined with your six
> > > ablation conditions is a clean and general evaluation methodology that
> > > could apply to any region-based compression scheme.
> > >
> > >
> > > Curve comparison results
> > >
> > > The B-spline vs Bezier vs Chebyshev comparison from v5.1 is very
> > > informative. The numbers are striking: on the irregular circle,
> > > B-splines achieve a full-image PSNR of 21.8 while Bezier gives 6.5 and
> > > Chebyshev just 2.0. The interior PSNR tells a similar story (47.3
> vs 7.0
> > > vs 1.7). You are right that some of the gap could reflect
> implementation

> > > differences, but a gap this large almost certainly reflects a genuine
> > > difference in suitability. B-splines' advantage comes from their local
> > > control property: moving one control point only affects a small
> section
> > > of the curve, whereas Chebyshev polynomials and single-piece Bezier
> > > curves are more global, so fitting errors propagate across the entire
> > > contour.
> > >
> > > For the paper, I would present these results but with appropriate care
> > > about the comparison being fair. Reviewers will want to know whether
> > > each method was given a similar parameter budget (similar number of
> > > control points or similar storage cost). If B-splines used 40 control
> > > points but Bezier used 10, the comparison is not apples-to-apples. I
> > > would suggest running the comparison across several images of varying
> > > complexity to show that the ranking is consistent, and normalising by
> > > the number of stored parameters or bytes so the reader can see
> that the
> > > advantage holds at equal cost. Including the original polynomial
> method
> > > from your first paper as a fourth row in that table would tie the
> > > progression together nicely.
> > >
> > >
> > > Code review
> > >
> > > Thanks for sharing your repository. I have looked through the v6.0
> > > notebook and the pipeline seems well-structured and clearly has come a
> > > long way from the original version. The shift to CIELAB, the
> integration
> > > of bilateral filtering, and the clean implementation of the planar
> model
> > > with numpy.linalg.lstsq are all solid. I have some specific
> observations
> > > that should be straightforward to address and that I think will
> give you
> > > noticeable improvements.
> > >
> > > repair_coverage bug: The "coverage = ..." and "return ..." lines
> at the
> > > bottom of the function are indented inside the "for _ in range(5)"
> loop,
> > > so the function always returns after a single iteration regardless of
> > > coverage. "Promoting" those two lines one level (so they sit
> outside the
> > > for loop) will let the iterative repair run to completion. The
> loop also
> > > uses a hardcoded range(5) instead of range(max_iters). Both are easy
> > > fixes. The concept behind this function is exactly what I had in mind
> > > when I suggested the uncoloured-pixel fallback, so you are on the
> right
> > > track, but it needs this small correction to work as intended.
> > >
> > > Once that is fixed, you will probably want to speed it up. The nested
> > > Python loops over every pixel will be slow on larger images. A
> > > vectorised approach using scipy.ndimage.binary_dilation on the painted
> > > mask to identify unpainted pixels adjacent to painted ones, then
> > > computing their fill colour via a small convolution, would make the

> > > repair step dramatically faster. OpenCV's cv2.inpaint could also
> work as
> > > a temporary debugging safeguard, but I would not rely on it as a core
> > > part of the reconstruction pipeline since a reviewer could question
> > > whether the quality is coming from your representation or from
> OpenCV's
> > > inpainting algorithm.
> > >
> > > Spline sampling during decompression: You set `n_samples = max(32,`
> `int(2`
> `* len(cx)))`. For large contours this can be too few - if a contour
> has a
> > > perimeter of 600 pixels but 80 control points, you evaluate 160 sample
> > > points, which leaves gaps that `cv2.fillPoly` bridges with straight line
> > > segments. Try setting `n_samples` proportional to the estimated arc
> length
> > > (e.g. sum of Euclidean distances between successive downsampled
> contour
> > > points), or simply `max(len(cx) * 4, perimeter_estimate)`. This is a
> quick
> > > change and I suspect it will visibly improve boundary smoothness,
> > > especially on the larger contours.
> > >
> > > Painter's algorithm sort key: You sort clusters by
> `len(cluster['pts'])`,
> > > which is the number of contour sample points rather than the actual
> > > region area. These are correlated but not the same - a region with a
> > > complex wiggly boundary might have many contour points but a small
> > > interior area, and vice versa. Sorting by the actual polygon area (you
> > > already have `contour_area_rc` in your code, or you could count filled
> > > pixels after `cv2.fillPoly`) would be more robust. This is a one-line
> > change.
> > >
> > > Spline fitting could give a potential quick win for compression:
> > > currently you downsample the contour with a fixed stride of 4 and then
> > > interpolate exactly through those points (`smooth=0.0`). This means the
> > > number of stored control points is always roughly `contour_length / 4`,
> > > which for long contours can be over 100 (I see entries with 140+
> control
> > > points in `imComp.txt`). An alternative that could substantially reduce
> > > file size: instead of downsampling first, pass the full contour to
> > > `splprep` with `smooth > 0`. The `s` parameter controls how much
> approximation
> > > error is allowed, and `scipy` will automatically determine the minimum
> > > number of knots and control points needed to meet that tolerance.
> Simple
> > > contours (ellipses, straight edges) would get very few control points
> > > while complex ones get more, so the representation becomes naturally
> > > adaptive. You would want to experiment with values of `s` to find the
> > > right trade-off between fidelity and compression. This single change
> > > could cut your file sizes significantly while maintaining similar
> > > reconstruction quality, and it is well worth investigating.
> > >
> > > CIELAB distance normalisation: the switch from HSV to CIELAB was a
> good
> > > move. The current distance metric divides each channel by 255,
> which is

> > > a reasonable first approximation but does not account for the
> different
> > > native ranges of the CIELAB channels. In OpenCV's 8-bit LAB
> > > representation, L* is scaled to 0-255 (from 0-100), and a* and b* are
> > > shifted to 0-255 (from roughly -128 to +127). A more perceptually
> > > meaningful approach would be to convert back to standard L-a-b ranges
> > > and compute a Euclidean distance in that space, which is at least
> closer
> > > to a delta-E style interpretation.
> > >
> > > $dE = \sqrt{dL^2 + da^2 + db^2}$
> > >
> > > The practical benefit is that your thresh parameter would then be in
> > > approximately delta-E units, which have a well-established perceptual
> > > meaning (delta-E < 1 is generally imperceptible, delta-E of about
> 2.3 is
> > > a just-noticeable difference). This would make the parameter more
> > > interpretable and easier to justify in the paper.
> > >
> > > Connectivity mismatch: your CCL uses 4-connectivity (the cross-shaped
> > > structuring element), but traceClusterEdges uses 8-connectivity (Moore
> > > neighbourhood tracing). This means the contour tracer might follow
> > > diagonal connections that the labelling step did not consider
> connected,
> > > which could contribute to subtle boundary artifacts. Making them
> > > consistent (either both 4 or both 8) would eliminate one potential
> > > source of error. This is a small thing but worth being aware of if you
> > > encounter hard-to-diagnose boundary issues.
> > >
> > > Clustering seed behaviour: looking at the clustering loop, the
> > > similarity mask is computed once per cluster iteration using the seed
> > > pixel's colour, rather than being iteratively re-estimated as the
> region
> > > grows. This means the initial seed's colour has a large influence on
> > > which pixels get accepted, which partly explains why the threshold
> > > parameter is so sensitive. A more robust approach would be to grow the
> > > region iteratively, updating the mean after each expansion pass and
> > > recomputing the similarity mask until convergence. That is a larger
> > > change and I would not prioritise it now, but it is worth noting
> for the
> > > paper's discussion of limitations and future work.
> > >
> > > pyrLevels: the colour-diversity-based pyramid level calculation in
> cell
> > > 4 is a nice idea, and I am glad you have started thinking about
> pyramid
> > > representations. You can revisit it when you get to the runtime
> > > optimisation stage.
> > >
> > > Redundant colour storage: the compressed file stores both the mean
> > > colour in the C line and the planar model coefficients in the M line.
> > > The mean colour is now redundant since the planar model subsumes it
> > > (when b=c=0, the planar model reduces to a constant, and `coef[:,0]`
> gives
> > > you the equivalent of the mean). You could simplify the format by
> > > dropping the colour from the C line and recovering it from the M
> line's

> > > constant term during decompression, to clean up the format.
> > >
> > >
> > > Other questions
> > >
> > > Should you start structuring the paper? Yes, now is a good time, and I
> > > would encourage you to start writing in parallel with further
> > > algorithmic improvements rather than waiting. You have some nice
> > > results, the core pipeline working, the ablation study, the curve-type
> > > comparison, the planar colour model, and the DCT hybrid proof of
> > > concept. That is a great foundation for a solid paper.
> > >
> > > In terms of structure, I would suggest: abstract, introduction
> > > (motivation, contributions), related work, method (segmentation,
> contour
> > > extraction, boundary encoding, interior colour model, reconstruction),
> > > experimental setup (datasets, metrics, baselines), results (ablation
> > > study, curve comparison table, rate-distortion curves by image
> category,
> > > hybrid DCT results), discussion (strengths, limitations, failure
> > > analysis), and conclusion.
> > >
> > > The act of writing will help you identify exactly which
> experiments you
> > > still need. You will find that structuring the results section
> naturally
> > > reveals gaps and scope for improvements. Do not wait until the
> algorithm
> > > is perfect before writing. No algorithm is ever perfect, and the paper
> > > should honestly discuss limitations and future work. The current
> > > limitations (thin-line artifacts on cartoons, texture loss on
> > > photographs without DCT residual) are completely reasonable things to
> > > acknowledge, especially since you can point to your ablation data to
> > > explain precisely why they occur and what would fix them.
> > >
> > > At this stage, I would optimise for publication quality rather than
> > > continuing to expand the algorithm in many directions at once.
> Focus on
> > > a short list of targeted improvements that are most likely to
> affect the
> > > final results meaningfully. This can be driven by your own
> curiosity as
> > > much as considerations about novelty and metric performance.
> > >
> > > Adaptive threshold metric: the most natural option is a measure of
> local
> > > colour variance or gradient energy. Before segmentation, compute the
> > > local standard deviation of colour values in a small window (e.g.
> 5x5 or
> > > 7x7) across the image. The image-wide mean or median of this measure
> > > gives you an overall complexity score. Low complexity (cartoons,
> logos)
> > > means you can usually afford a higher threshold, because there is
> little
> > > risk of merging across a genuine boundary. High complexity
> (photographs)
> > > means a lower threshold to avoid over-merging across edges and fine

> > > structure.

> > >

> > > A more principled version (for future work if not now) would be to

> > > replace the scalar Euclidean threshold with a Mahalanobis-type

> distance

> > > using a locally estimated covariance. That would let the acceptance

> > > criterion adapt automatically by channel and by region statistics,

> which

> > > is closer to what you ultimately want.

> > >

> > > One thing to be aware of is that the threshold interacts with the

> > > bilateral filter strength. A stronger bilateral filter (higher

> > > sigmaColor) removes more texture before clustering, which has a

> similar

> > > effect to raising the threshold. You have two adjustable dials that do

> > > related things, so it is worth studying their interaction, perhaps by

> > > running a small grid search over (sigmaColor, thresh) pairs on a few

> > > representative images and plotting the resulting PSNR. That analysis

> > > would also be good material for the paper.

> > >

> > > Metric for deciding region-based vs DCT encoding: I would frame

> this as

> > > a local rate-distortion decision rather than a single ad-hoc

> complexity

> > > score. The core question is: for each region, should you store only

> > > spline boundaries + planar colour model, or should you also add a

> > > DCT-coded residual? The answer depends on whether the extra bytes

> spent

> > > on the residual are "worth it" in terms of the quality

> improvement. For

> > > any given region, you have (at least) two encoding options. Option 1

> > > (custom only) costs R_{custom} bytes and produces a reconstruction with

> > > distortion D_{custom} (measured e.g. as MSE against the original).

> Option

> > > 2 (hybrid) costs R_{hybrid} bytes (which will be larger, since you are

> > > storing the spline/planar data plus the DCT residual) but produces a

> > > lower distortion D_{hybrid} . The standard way to compare these is to

> > > define a combined cost for each option:

> > >

> > > $J_{\text{custom}} = D_{\text{custom}} + \lambda * R_{\text{custom}}$

> > > $J_{\text{hybrid}} = D_{\text{hybrid}} + \lambda * R_{\text{hybrid}}$

> > >

> > > and choose whichever gives the smaller J. The lambda parameter

> controls

> > > the rate-distortion trade-off: a large lambda penalises bytes heavily,

> > > so the method will prefer the cheaper option unless the quality gain

> > > from adding a residual is very large; a small lambda means you

> care more

> > > about quality and will more readily spend extra bytes on the DCT

> > > residual. This is exactly how modern video and image codecs (H.264,

> > > H.265, AV1) decide between different block sizes, prediction

> modes, and

> > > transform types internally. You do not need a perfect

> implementation of

> > > this for the paper, even a simplified version where you estimate D

> from

> > > the planar-fit residual variance and estimate R from the number of

> > > control points plus DCT coefficient count would be a principled and
> > > defensible approach. The lambda value can be set globally (one
> > > value for
> > > the whole image) and tuned to target a desired overall bitrate or
> > > quality level.
> > >
> > > If you want a simpler first version before implementing a full
> > > rate-distortion framework, a good proxy would be as follows. After
> > > computing the planar fit for each segmented region, measure: (i)
> > > residual MSE per pixel, i.e. $\text{mean}((\text{original} - \text{planar_fit})^2)$
> > > (ii) high-frequency residual energy, for example the energy in AC DCT
> > > coefficients or the gradient/Laplacian energy of the residual
> > > (iii) estimated coding cost of the spline + planar representation
> > > (iv) estimated coding cost of adding a quantised DCT residual.
> > >
> > > A simple decision rule: compute $\text{residual_std} = \sqrt{\text{residual_var}}$ per
> > > region and also a texture score:
> > >
> > > $\text{texture_score} = \text{high_frequency_residual_energy} / \text{total_residual_energy}$
> > >
> > > Regions with low residual MSE and low texture score stay in the
> > > spline/planar mode. Regions with high residual MSE and high texture
> > > score get a DCT residual. You could even make it a three-tier system:
> > > very low residual gets planar only; moderate gets a higher-order
> > > polynomial interior model (e.g. quadratic:
> > >
> > > $\text{colour}(x,y) = a + b*x + c*y + d*x^2 + e*y^2 + f*x*y$
> > >
> > > which is still very compact at 6 floats per channel); high gets DCT
> > > residual.
> > >
> > > This per-region adaptive decision is a strong contribution for the
> > > paper, because it means your codec automatically adjusts its encoding
> > > strategy to local image content. Most standard codecs apply a single
> > > strategy uniformly. Your method's ability to treat a cartoon
> > > character's
> > > body differently from the textured grass it is standing on is a
> > > genuine
> > > advantage worth highlighting.
> > >
> > > I would not use exactly the same metric for this decision and for the
> > > segmentation threshold. They are related but they solve different
> > > problems: one is a clustering decision made early in the pipeline; the
> > > other is a coding-mode decision made after you already have a
> > > candidate
> > > region model.
> > >
> > > You are right though that there is a connection - regions with high
> > > gradient energy before segmentation tend to be the same ones that will
> > > have high residual variance after planar fitting. So there may be a
> > > unified complexity measure that informs both decisions, and that would
> > > be an elegant result if you can demonstrate it.
> > >
> > >
> > > Further suggestions
> > >
> > > For the Bugs Bunny boundary problem specifically (boundary PSNR of 9.4

> > > is very poor), the thin dark strokes are 1-2 pixels wide, and when the
> > > contour is downsampled by stride 4 and then fitted with a smooth
> > > B-spline, those structures get smeared away. The "thin dark stroke
> > > pathway" is still on your outstanding list and I think it would be
> > > high-impact for cartoon images: run a gradient magnitude filter (or
> > > Canny) before segmentation to identify thin dark lines, and store
> these
> > > as a separate overlay layer using chain codes or run-length encoding
> > > (Freeman chain codes would be natural here since they are designed for
> > > encoding contour paths efficiently). Dark strokes on cartoons are
> > > inherently one-dimensional structures and compress very well. During
> > > reconstruction, draw the overlay on top of the region-filled
> image. This
> > > would make a visible difference to your cartoon results and is an
> > > interesting content-adaptive feature to discuss in the paper.
> > >
> > > For rate-distortion comparisons, you will need to measure the
> total byte
> > > count of your compressed output, including both the region/spline data
> > > and the quantised DCT residual coefficients. The current DCT payload
> > > dictionary stores raw numpy arrays in memory, which does not
> represent a
> > > file size. You will need to serialise everything and measure bytes to
> > > make fair comparisons against JPEG and PNG. Plotting rate-distortion
> > > curves (bits per pixel vs PSNR) by image category should be a core
> > > figure in the paper. This ties into the encoding step you have
> planned -
> > > once you implement binary encoding with entropy coding for both the
> > > spline coefficients and the DCT residual, the numbers should be much
> > > more competitive than the current text-based format.
> > >
> > >
> > > Priorities
> > >
> > > In terms of where to focus next, I suggest
> > >
> > > 1. Fix the immediate code issues identified above (repair_coverage
> bug,
> > > spline sampling, Painter's sort key).
> > >
> > > 2. Start the paper outline and begin writing the method section(s)
> while
> > > the implementation details are fresh. Do both writing and
> development in
> > > parallel from this point.
> > >
> > > 3. Experiment with smooth > 0 in splprep and measure its impact on
> both
> > > reconstruction quality and file size. This could be your single
> biggest
> > > compression improvement.
> > >
> > > 4. Formalise the per-region DCT decision metric and integrate it into
> > > the pipeline.
> > >
> > > 5. Implement binary encoding for the compressed file and measure
> actual

> > > compression ratios. This is essential before you can make any
> > > rate-distortion claims in the paper.
> > >
> > > 6. Leave Gaussian pyramids and full entropy-coding refinement until
> > > after the core experimental story is stable, but do move to a real
> > > binary serialisation and actual byte-count measurement before making
> > > compression-ratio claims.
> > >
> > > The work is progressing really well and you are at a genuinely good
> > > stage. You now have a clear method, a clean ablation story,
> identifiable
> > > failure modes, and a plausible next step that directly addresses the
> > > current weaknesses. Excellent progress!
> > >
> > >
> > > Best wishes,
> > > Chris
> > >
> > >
> > > On 30/03/2026 01:47, D.M. Fisher wrote:
> > >> Hi Chris,
> > >>
> > >> No worries at all, totally understand and my responses have been
> > delayed as well as I work through this project. I've got plenty to work
> > on with it in the interim. Hope things start to settle out a bit as you
> > move into Easter term soon!
> > >>
> > >> I really appreciate both the positive and constructive feedback
> > again, it's exciting as I start to see this algorithm coming together
> > and working on more and more complex images. The recommendations have
> > been excellent for learning some new things beyond what we covered in
> > our CV class, and several of them have led to step improvements in my
> > outputs. Your explanation of the buckets of errors makes a lot of sense
> > to me and as I did the ablation test and made some of the updates we've
> > discussed in this last round, I've seen them prove to be more evident.
> > >>
> > >> Addressing your suggested priorities in order:
> > >> (1) Ablation study to quantify error sources:
> > >> I was interested in investigating a few more combinations of
> > decompression methods in addition to the ones you recommended and tested
> > the following:
> > >> A. Exact contours + mean color of cluster
> > >> B. B spline + mean color of cluster
> > >> C. Exact contours + original image mask
> > >> D. Exact contours + planar parametric region
> > >> E. B spline + planar parametric region
> > >> F. B spline + original image mask
> > >>
> > >> I split them out into full image, boundary regions, and interior
> > regions (which I determined based on my clustering determination earlier
> > in the code). To have some metrics to analyze the different combinations
> > on this I implemented PSNR, SSIM, and MAE. Below you can see first my
> > results for the penguin test, bugs bunny, and then the mountain. I also
> > attached the decompressed images for these.
> > >>
> > >> Penguin:
> > >> A exact+mean:

```
> > >> full PSNR=25.397, SSIM=0.8892
> > >> boundary MAE=17.426, PSNR=20.193, SSIM=0.8783
> > >> interior MAE=5.144, PSNR=29.145, SSIM=0.8957
> > >> B spline+mean:
> > >> full PSNR=22.157, SSIM=0.8659
> > >> boundary MAE=23.877, PSNR=15.921, SSIM=0.8001
> > >> interior MAE=5.144, PSNR=29.145, SSIM=0.8861
> > >> C exact+original:
> > >> full PSNR=43.417, SSIM=0.9948
> > >> boundary MAE=1.790, PSNR=37.791, SSIM=0.9935
> > >> interior MAE=0.301, PSNR=48.133, SSIM=0.9952
> > >> D exact+planar:
> > >> full PSNR=25.772, SSIM=0.8884
> > >> boundary MAE=16.857, PSNR=20.479, SSIM=0.8865
> > >> interior MAE=4.957, PSNR=29.701, SSIM=0.8927
> > >> E spline+planar:
> > >> full PSNR=22.334, SSIM=0.8650
> > >> boundary MAE=23.347, PSNR=16.022, SSIM=0.8080
> > >> interior MAE=4.881, PSNR=29.749, SSIM=0.8831
> > >> F spline+original:
> > >> full PSNR=29.126, SSIM=0.9863
> > >> boundary MAE=4.112, PSNR=22.176, SSIM=0.9674
> > >> interior MAE=0.301, PSNR=48.133, SSIM=0.9908
> > >>
> > >> Bugs bunny:
> > >> A exact+mean:
> > >> full PSNR=31.985, SSIM=0.9200
> > >> boundary MAE=6.752, PSNR=28.212, SSIM=0.9866
> > >> interior MAE=3.244, PSNR=35.755, SSIM=0.8875
> > >> B spline+mean:
> > >> full PSNR=14.711, SSIM=0.6037
> > >> boundary MAE=47.949, PSNR=9.438, SSIM=0.0635
> > >> interior MAE=3.246, PSNR=35.747, SSIM=0.8513
> > >> C exact+original:
> > >> full PSNR=38.512, SSIM=0.9373
> > >> boundary MAE=2.897, PSNR=36.273, SSIM=0.9975
> > >> interior MAE=1.989, PSNR=39.955, SSIM=0.9081
> > >> D exact+planar:
> > >> full PSNR=32.207, SSIM=0.9203
> > >> boundary MAE=6.336, PSNR=28.835, SSIM=0.9881
> > >> interior MAE=3.547, PSNR=35.148, SSIM=0.8872
> > >> E spline+planar:
> > >> full PSNR=14.748, SSIM=0.6039
> > >> boundary MAE=47.597, PSNR=9.479, SSIM=0.0651
> > >> interior MAE=3.551, PSNR=35.136, SSIM=0.8509
> > >> F spline+original:
> > >> full PSNR=38.501, SSIM=0.9373
> > >> boundary MAE=2.897, PSNR=36.273, SSIM=0.9975
> > >> interior MAE=1.991, PSNR=39.933, SSIM=0.9078
> > >>
> > >> Mountain:
> > >> A exact+mean:
> > >> full PSNR=23.838, SSIM=0.7805
> > >> boundary MAE=12.921, PSNR=23.560, SSIM=0.6818
> > >> interior MAE=11.470, PSNR=24.038, SSIM=0.8464
> > >> B spline+mean:
> > >> full PSNR=17.700, SSIM=0.6240
```

> > >> boundary MAE=25.980, PSNR=14.526, SSIM=0.3237
> > >> interior MAE=11.655, PSNR=23.511, SSIM=0.8271
> > >> C exact+original:
> > >> full PSNR=37.228, SSIM=0.9498
> > >> boundary MAE=3.457, PSNR=34.839, SSIM=0.9401
> > >> interior MAE=1.505, PSNR=40.258, SSIM=0.9534
> > >> D exact+planar:
> > >> full PSNR=26.415, SSIM=0.8140
> > >> boundary MAE=10.172, PSNR=25.290, SSIM=0.7519
> > >> interior MAE=7.260, PSNR=27.399, SSIM=0.8549
> > >> E spline+planar:
> > >> full PSNR=24.255, SSIM=0.7272
> > >> boundary MAE=13.801, PSNR=21.820, SSIM=0.5526
> > >> interior MAE=7.259, PSNR=27.399, SSIM=0.8475
> > >> F spline+original:
> > >> full PSNR=31.013, SSIM=0.9300
> > >> boundary MAE=5.093, PSNR=27.424, SSIM=0.8928
> > >> interior MAE=1.508, PSNR=40.221, SSIM=0.9520
> > >>
> > >> The ablation results align closely with the three error buckets you
> > described. Using exact contours with original pixel values produced
> > near-perfect reconstructions across all test cases, which shows me that
> > segmentation is largely correct and not the dominant source of error.
> > >>
> > >> For simpler images like the penguin, the primary degradation looks
> > like it comes from boundary approximation based on the drop in boundary
> > PSNR when introducing B-splines. This effect appears much more
> > pronounced with Bugs Bunny, where the B-spline boundaries fail to
> > capture thin structures and sharp edges, resulting in severe boundary
> > error despite otherwise accurate segmentation.
> > >>
> > >> For natural images like the mountain, the main limitation looks like
> > the region appearance model. While planar parametric fitting improved
> > interior reconstruction relative to mean color, there is still a large
> > gap compared to using original pixel values, which tells me that a
> > constant or low-order model can't capture high-frequency texture and
> > shading.
> > >>
> > >> Overall, the results seem to validate the distinction between
> > segmentation error, boundary approximation error, and region appearance
> > error, and show that the dominant failure mode is highly dependent on
> > image type.
> > >>
> > >> (2) Upgrade interior color model to parametric:
> > >> Complete per (1) as part of the ablation testing. It seems like a
> > clear improvement over using the mean visually and the results agree.
> > >>
> > >> (3) Add bilateral pre-filtering:
> > >> I implemented this using OpenCV's package and included it in the
> > ablation testing. I saw some slight improvement before and after.
> > >>
> > >> (4) Implement PSNR/SSIM metrics:
> > >> As mentioned, completed for the ablation testing, in addition to MAE.
> > >>
> > >> (5) Revisit encoding and compression ratio comparisons:
> > >> Not yet done, but I intend to perform this in the next update.
> > >> Something you mentioned a while ago that I wanted to revisit, as we are

> > now at the point where the work is starting to come together a bit, is
> > with regard to hybrid compression. I tested out taking the difference
> > between my compressed image and the original, and compressing it using
> > the DCT. This yielded a great result. Something I want to look into is
> > using a metric (such as a residual energy or gradient-based approach) to
> > analyze each region and more decisively decide to encode it using my
> > method or the DCT. Then I can store both of these in a file using a
> > better encoding schema and see what the ultimate compression ratio looks
> > like. Do you have any suggestions on the best metric to make this
> decision?
> > >>
> > >>
> > >> Additional updates/improvements:
> > >> In v5.1 I set up an ablation test for B-splines vs Bezier vs
> > Chebyshev. I think even though I saw reasonable results before for
> > Chebyshev, that was a while back when I was doing much simpler and
> > smaller images and this method seems to struggle to accurately represent
> > the images of higher complexity that I am now working with. Bezier
> > curves are slightly better than Chebyshev, but from this testing, I am
> > seeing them to be far inferior to B-splines. I also recognize that some
> > of this could be biased based on how effectively I've implemented these
> > methods. I included one set of results below for the irregular circle.
> > >> B-spline:
> > >> full PSNR=21.820, SSIM=0.9543
> > >> boundary MAE=23.974, PSNR=14.164, SSIM=0.8474
> > >> interior MAE=0.971, PSNR=47.349, SSIM=0.9789
> > >> Bezier:
> > >> full PSNR=6.537, SSIM=0.5800
> > >> boundary MAE=110.618, PSNR=4.737, SSIM=0.0764
> > >> interior MAE=62.004, PSNR=7.024, SSIM=0.6780
> > >> Chebyshev:
> > >> full PSNR=2.044, SSIM=0.2125
> > >> boundary MAE=123.217, PSNR=4.242, SSIM=0.0180
> > >> interior MAE=188.439, PSNR=1.701, SSIM=0.2680
> > >>
> > >> I implemented CIELAB as the color schema vice HSV and saw some minor
> > improvements.
> > >>
> > >> I identified that a lot of the artifacting, specifically with
> > regards to the large white blotches, were due to all pixels in the
> > decompressed image not being assigned to clusters (and the default image
> > is white so they stay white). I implemented a function that requires at
> > least 98% of pixels be assigned via nearest neighbor, which showed
> > significant improvements in my output.
> > >>
> > >> I shared the Github with you. Hopefully this offers some insight
> > into where I am at with everything. Please excuse my formatting, as
> > through the many updates I have made, I haven't kept all of my variable
> > names and comments as clean as I would for a finalized version.
> > >>
> > >>
> > >> Additional Questions:
> > >> Based on the current results, do you think the work is approaching a
> > stage where it would be valuable to begin structuring the paper and
> > formal evaluation, or would you recommend continuing to prioritize
> > algorithmic improvements before shifting focus?
> > >>

> > >> I have a hyperparameter I am calling Threshold, which indicates the
> > tolerance of difference permissible for two regions to be considered the
> > same cluster. I am finding experimentally that appropriate values that
> > yield good results range from .15 to .25. I think it would be beneficial
> > to make this value adaptive based on some metric in the image. What do
> > you think might be a good metric to base this off of? I think this could
> > possibly be the same metric that I use to decide which regions to encode
> > with my algorithm or the DCT, so I am considering some of the ideas I
> > mentioned above In that paragraph.

> > >>

> > >>

> > >> Next steps/outstanding recommendations I am tracking:

> > >> Revisit deterministic hybrid DCT/B-spline encoding method (in
> > previous versions like v4.0 I experimented with hybrid encoding)

> > >>

> > >> Implement adaptive boundary model

> > >>

> > >> Replace Painter's algorithm with topological hierarchy such as
> > Suzuki's algorithm

> > >>

> > >> Add explicit "thin dark stroke" pathway for cartoons

> > >>

> > >> Gaussian pyramid to reduce runtime complexity (you may see I started
> > to implement a portion of this)

> > >>

> > >> Implement entropy coding method

> > >>

> > >>

> > >>

> > >>

> > >> Thanks again for your advising!

> > >>

> > >> Kind regards,

> > >> Daniel

> > >>

> > >> -----Original Message-----

> > >> From: Chris Town <cpt23@cam.ac.uk>

> > >> Sent: Saturday, March 14, 2026 12:41

> > >> To: D.M. Fisher <dmf41@cantab.ac.uk>

> > >> Subject: Re: Revisiting 2021 Computer Vision Project for Possible
> > Publication

> > >>

> > >> Hi Daniel,

> > >>

> > >> Sorry for my slow response, we are getting close to the end of term
> > and work always seems to pile up around this time.

> > >>

> > >> Well done on making so many improvements! It is great to see your
> > segmentation working on higher resolution images. I'm glad the B-splines
> > approach is working well and that region closure is significantly
> > improved. The leap to 256x256 images is a major step forwards.

> > >>

> > >> To answer your question regarding the source of the remaining
> > artefacts, it really depends on the complexity of the image, as shown in
> > your examples.

> > >>

> > >> For simple subjects (like the balloons, the irregular outline, and

> > the penguin), your custom reconstruction is already fairly close. The
> > overall shape and colour regions are well-captured, but if you look
> > carefully at the edges, especially around the penguin's body outline and
> > beak, there is visible jaggedness and slight boundary misalignment.
> > >> These are spline-fitting artefacts rather than segmentation failures.
> > >> They are the residual errors you would expect from a B-spline that
> > doesn't perfectly track a sharp corner or concavity.
> > >>
> > >> However, many of the artefacts seen in the more complex images are
> > caused by inaccuracies and limitations in the colour segmentation phase,
> > not the B-spline approximation. A second major issue is the interior
> > appearance model: even perfect segmentation and perfect splines will not
> > recover moon texture, grass texture, cloud detail, or fabric detail if
> > each region is filled only with a mean colour.
> > >>
> > >> In the Bugs Bunny cartoon, thin features like the whiskers and the
> > dark outlines are severely broken or missing. Euclidean/HSV thresholding
> > breaks the continuity of thin lines. Large interior regions of the body
> > are effectively collapsing into sparse fragments. That does not look
> > like simple boundary misplacement, it looks more like incorrect
> > segmentation, merging, or unstable treatment of low-saturation greys,
> > possibly combined with an over-aggressive region model.
> > >>
> > >> Likewise, in the football, mountain, and moon images, the dominant
> > issue is that textured or smoothly varying areas are being replaced by a
> > small number of flat regions. That is mostly a segmentation-plus-region-
> > appearance problem, not a contour-only problem.
> > >> A perfect spline around the moon still cannot recover the lunar
> > texture if the region interior is represented only by a mean colour.
> > >>
> > >> Your original paper already framed the method as best suited to
> > images with large continuous colour regions, and these new results are
> > consistent with that.
> > >>
> > >> I would split the current error sources into three "buckets":
> > >>
> > >> 1. Segmentation / region assignment error: wrong pixels placed into
> > the wrong region, or thin/low-contrast structures merged away.
> > >>
> > >> 2. Boundary approximation / rasterisation error: good region
> > assignment, but fitted contours round corners, drift, or fail to align
> > exactly with the raster grid.
> > >>
> > >> 3. Region appearance model error: even with correct regions and
> > correct boundaries, a single mean colour per region is too weak for
> > gradients, shading, and texture.
> > >>
> > >> From the examples, bucket 3 is the biggest issue on natural
> > images, bucket 1 is on cartoon images with fine lines, and bucket 2 is
> > evident around high-curvature corners and thin structures.
> > >>
> > >> A useful next step is to measure those three error sources
> > separately.
> > >> Right now you are inferring their roles visually, but you can answer
> > the question empirically with a small ablation study, which is the right
> > approach for a publication.
> > >>

> > >> I suggest you run these four reconstructions on the same segmented
> > image:

> > >>

> > >> A. Exact traced contours + current region fill model: no spline
> > fitting, fill using the exact raster contour from the segmentation
> > stage. This removes spline error.

> > >>

> > >> B. Spline contours + current region fill model: the difference
> > between A and B isolates boundary-model error.

> > >>

> > >> C. Exact traced contours + original per-pixel colours inside each
> > predicted region: any remaining error in C mainly reflects mask or
> > topology issues.

> > >>

> > >> D. Exact traced contours + a better region model: for example, per-
> > region planar colour fit (see below) instead of mean colour. This tells
> > you how much is due to the constant-colour interior assumption.

> > >>

> > >> Compute error separately in a narrow band around region boundaries
> > and in region interiors. If boundary-band error is small but interior
> > error is large, then improving splines further will not solve the main
> > problem. That is the experiment I would prioritise before spending much
> > more time on encoding.

> > >>

> > >> On the modelling side, I would make two immediate changes:

> > >>

> > >> - Upgrade the interior colour model. A single mean colour is too
> > weak for sky, clouds, moon surfaces, skin, grass, and textured fabrics.
> > The lowest-complexity improvement is to fit each channel in each region
> > to a planar parametric model such as

> > >>

> > >> $\text{colour}(x,y) = a + b*x + c*y$

> > >>

> > >> for each colour channel separately, where x,y are pixel coordinates,
> > a is the base colour intensity (similar to your current mean, but
> > adjusted for the gradient), b models how the colour changes horizontally
> > and c how the colour changes vertically.

> > >> To find the optimal coefficients (a, b, c) that minimise the squared
> > error across all pixels in a given region, we can set this up as a
> > standard linear least-squares problem and you can use
> > "numpy.linalg.lstsq" for this.

> > >> That representation is still extremely compact: rather than one
> > floating point number per channel for the mean, you are now saving 3
> > floats per channel for a, b, c . However, it can model smooth gradients
> > and shading far better than a constant fill. For the moon and mountain
> > examples ("bucket 3" errors), that one change may give a bigger visual
> > gain than any further refinement of the contour model. Moreover, this
> > strengthens the resolution-independent nature of the whole algorithm
> > because then planar fit can be evaluated at any pixel density, just like
> > the B-spline boundaries.

> > >>

> > >> - Make the boundary model adaptive rather than universally smooth.
> > >> B-splines are a good choice overall, but they will naturally soften
> > sharp corners. For cartoons and logos, I would detect high-curvature or
> > corner-like contour points and either split the contour there, increase
> > knot multiplicity there, or fall back to a polygonal or chain-code style
> > encoding for that region. Use splines for smooth boundaries, but do not

> > force smoothness onto genuinely angular geometry.
> > >>
> > >> Some other suggestions/ ideas:
> > >>
> > >> - Keep the scanline fill. That was a very good change.
> > >>
> > >> - Upgrade to a topological hierarchy: sorting regions by area
> (Painter's
> > >> algorithm) is a clever quick fix, but it can hide structural
> > mistakes and cause incorrect stacking if a small background region peers
> > through a hole in a larger foreground object. Look into established
> > contour retrieval algorithms (such as Suzuki's algorithm) that
> > automatically build a parent-child hierarchy tree to explicitly define
> > outer boundaries and holes.
> > >>
> > >> - As a fallback, any pixel that remains uncoloured after all regions
> > are rendered should be assigned the colour of the nearest filled pixel
> > (or nearest region centroid). This doesn't fix remaining segmentation
> > problems, but it eliminates any visually jarring artifacts immediately
> > and makes the results much more presentable.
> > >>
> > >> - Implement pre-processing smoothing: natural images are incredibly
> > noisy compared to your initial test cases. I strongly recommend applying
> > a bilateral filter to the image before clustering. This will smooth out
> > minor colour variations while strictly preserving sharp edges, giving
> > your CCL algorithm much cleaner, larger continuous regions to work with.
> > >> Bilateral filtering preserves sharp edges while smoothing texture,
> > so it is a better choice than the Gaussian blur I suggested before.
> > >>
> > >> - Revisit colour space again for low-saturation content. Your hue
> > downweighing is sensible, but Bugs Bunny suggests that greys and near-
> > greys are still a weak spot. A perceptually motivated space such as
> > CIELAB or OKLab may behave better there than HSV.
> > >>
> > >> - Add an explicit "thin dark stroke" pathway for cartoons. The ears,
> > whiskers, outlines, and fingers are exactly the kind of structures that
> > disappear when region merging is even slightly too aggressive. A very
> > lightweight edge/stroke overlay layer could dramatically improve cartoon
> > reconstructions.
> > >>
> > >> - For photos, strongly consider a hybrid codec: region/spline base
> > layer plus a residual stream. Otherwise the method will continue to
> > struggle on texture-heavy scenes for reasons that are fundamental, not
> > merely implementation-related. As I suggested before, the difference
> > (reconstruction residuals) could then be compressed separately using a
> > standard method like a wavelet or discrete cosine transform (as in JPEG)
> > at a chosen compression factor/loss level.
> > >>
> > >> On runtime, I agree with your prioritisation: correctness first,
> > then speed. But once the error source ablation is done, I would move
> > next to replacing any remaining repeated whole-image scans with local
> > region growing or component-wise processing. Your original method's
> > complexity came from repeated image-wide passes and recursive
> > reachability logic.
> > >> Your CCL shift already breaks that dependence; the next performance
> > gain is to make the rest of the pipeline equally local, so that work
> > scales with region size or contour length rather than with the full

> > image area every time.
> > >>
> > >>
> > >> For publication framing, I would not position this as a universal
> > still-image compressor trying to beat JPEG or PNG on arbitrary
> > photographs. Instead, frame it around "semantic image compression" or
> > "automated raster-to-vector (SVG) conversion." Because your algorithm
> > defines images using mathematics, it is inherently vector-like and
> > somewhat scale-invariant (resolution-independent) upon decompression,
> > which is a major advantage standard formats cannot offer.
> > >>
> > >> For the paper itself, it would be interesting to include:
> > >>
> > >> - An ablation table comparing polynomial, Chebyshev, Bézier, and B-
> > spline boundary models
> > >>
> > >> - Rate-distortion curves by image category rather than one pooled
> > average
> > >>
> > >> - Runtime scaling with resolution
> > >>
> > >> - Failure-case analysis by category
> > >>
> > >> There should also be a clarification that natural-image texture is
> > not expected to be handled well without an interior model richer than a
> > mean colour.
> > >>
> > >> For future refinements, being able to look at your code would be
> > useful.
> > >> If you'd like to share your code, you could use a GitHub repository
> > - it does not have to be public, i.e. you can have a private repository
> > and then share access selectively.
> > >>
> > >> In terms of where to focus next, I'd suggest you prioritise as
> > follows:
> > >> (1) run the ablation study (conditions A–D) to quantify the three
> > error sources, (2) upgrade the interior colour model to a planar
> > parametric model, (3) add bilateral pre-filtering, (4) implement PSNR/
> > SSIM metrics so you can measure progress quantitatively, (5) then
> > revisit encoding and compression ratio comparisons.
> > >>
> > >> I am really pleased with your progress. The move away from the
> > original recursive reachability bottleneck and global polynomial fitting
> > towards CCL, ordered contours, deterministic filling, and spline-based
> > boundaries makes the work much closer to being publication-ready.
> > >>
> > >> Well done!
> > >>
> > >> Best wishes,
> > >> Chris
> > >>
> > >>
> > >> On 04/03/2026 03:04, D.M. Fisher wrote:
> > >>> Chris,
> > >>>
> > >>> Fortunately the snow has died down a bit here and it is starting to
> > warm up a bit! I fear cold rain is the worst of both worlds so hopefully

> > you've gotten some snow and things will warm up there soon too. It has
> > been several years since I was last in Cambridge but I remember the
> > summer was beautiful.
> > >>>
> > >>> Thank you so much for the thorough feedback, I appreciate the
> > thoughtful insights. It definitely gave me a lot of interesting ideas to
> > try out and has very much led me down the path of relearning a lot of
> > these topics which is definitely a part of what I was hoping to get out
> > of this project.
> > >>>
> > >>>
> > >>>
> > >>>
> > >>> I have made the following progress:
> > >>>
> > >>> Polynomial Fitting:
> > >>> -Identified instability you discussed in numpy.polyfit (Vandermonde
> > matrix + power basis), particularly at higher degrees, contributing to
> > Runge-type oscillations.
> > >>>
> > >>> Evaluated the following fitting approaches:
> > >>> -Polynomial fitting (original method) with capping max degree -->
> > >>> significant improvement in compression ratio and reconstruct
> stability
> > >>> -Chebyshev polynomials --> moderate improvement in numerical
> > stability over polynomial fitting -B-splines --> very noticeable
> > improvement in ability to reconstruct with some tradeoff of less
> > compression, this is my chosen method now!
> > >>> -Piecewise cubic Bézier segments --> I expected this to show the
> > best improvement due to its frequent use in vector-graphics style
> > parametric curves, but B-splines still performed better thus far.
> > Slightly better compression than B-Splines though.
> > >>> -Fourier descriptors --> Decided not to go this route, as it still
> > >>> uses global approach and is less suitable for sharp corners
> > >>>
> > >>> Contour Preprocessing:
> > >>> -Ensured contour vertices are correctly ordered prior to regression
> > to avoid ill-conditioned fits caused by unordered boundary points.
> > >>>
> > >>> Reconstruction/Rasterization:
> > >>> -Replaced previous 9-neighbor flood-style filling (which was
> > vulnerable to small rasterization gaps from imperfect curve fits) with a
> > deterministic scanline polygon fill.
> > >>> -Contours are explicitly closed prior to filling.
> > >>> -Reconstruction now relies on an even-odd rule scanline fill rather
> > than evaluating polynomial functions over the grid.
> > >>> -Implemented a priority-based rasterization (Painter's Algorithm):
> > regions are sorted by area and rendered largest to smallest, allowing
> > nested regions (holes) to be handled implicitly through overwrite rather
> > than explicit boolean subtraction.
> > >>> -This change significantly improved topological robustness and
> > eliminated the previous boundary leakage artifacts.
> > >>> -Attempted explicit hole-aware region encoding, where inner
> > contours are stored separately and excluded during reconstruction rather
> > than relying on painter-style overwriting. Showed significant
> > degradation to compression ability and adds a lot of complexity to
> > compression process.

> > >>>
> > >>> Runtime improvements:
> > >>> I made several modifications to my cluster assignment function
> > leveraging NumPy vectorization to improve runtime:
> > >>> -Replaced the nested seed search loop used to locate the next
> > cluster start with a direct lookup of the first unvisited pixel.
> > >>> -Replaced the per-pixel similarity check loop with a vectorized
> > circular HSV distance computation across the image, rather than
> > evaluating the similarity metric individually inside nested loops.
> > >>> -Started using a running counter for remaining unassigned pixels
> > instead of recomputing np.count_nonzero(visited) each iteration.
> > >>>
> > >>> Color Space Modifications:
> > >>> -I had already implemented a circular HSV function but added in a
> > negative weighting for hue when saturation is low as you recommended and
> > also saw improvements from that.
> > >>>
> > >>>
> > >>> Main Focus:
> > >>>
> > >>> I really want to solidify a precise algorithm that can reliably and
> > robustly compress and decompress these images (mostly doing 256x256
> > now). Specifically shifting to B-Splines and enforcing region closure
> > showed great improvements. I am continuing to refine these methods a bit
> > to make sure I have implemented them optimally since I am still seeing a
> > degree of artifacting that I want to reduce. However, since we have
> > revisited this work, I have seen astronomical improvements which is
> > really encouraging. The algorithm now handles test cases that I couldn't
> > even attempt to run when I did this as a student with ease and I have
> > now been testing on landscapes and cartoons. I've attached a few images
> > to this email to give you an idea of what it can and can't handle
> right now.
> > >>>
> > >>>
> > >>>
> > >>>
> > >>> For my next steps I intend to:
> > >>>
> > >>> -Reduce artifacting...as you can see in some of the more complex
> > images I attached, there are still significant issues, though it is at
> > least quite recognizable now.
> > >>>
> > >>> -Reducing runtime complexity --> since the changes I mentioned in
> > my last email, I have primarily been focused on getting the images to
> > accurately compress/decompress and haven't spent as much time on ideas
> > you mentioned like implementing coarse-to-fine image pyramid processing,
> > using Gaussian pyramids, parallelizing region processing, etc. but
> > intend to later on as well.
> > >>>
> > >>> -Revisit encoding schemas --> this has been done a lot so I am less
> > concerned about nailing that down prior to confirming the validity of my
> > algorithm and it is actually a bit easier right now to have my
> > compressed file be in a format that is more readable to help me debug. I
> > intend to implement this later on though so I don't sell my algorithm
> > short when compared to others and can effectively evaluate it. With the
> > shift to B-splines, my method is generally not beating JPEG and often

> > not PNG, depending on how detailed the image is.
> > >>>
> > >>> -Test against datasets/quantitative metrics --> I appreciate all of
> > the recommended testing sets and metrics you sent. I have made a dataset
> > of images that allow me to gradually step up the difficulty and attempt
> > different challenges like lots of holes, overlapping figures, concentric
> > images, etc. to verify components I think the algorithm previously
> > struggled with and it now seems to be performing fairly well on those.
> > Now I intend to move on to more widely accepted datasets and use cases
> > you mentioned (ex. cartoons, space) as I am getting more confident in
> > the algorithm. I will also keep these metrics in mind for the evaluation
> > portion of the paper when I start rewriting that portion.
> > >>>
> > >>> -Literature search/paper framing --> I appreciate the
> > recommendations on extending my literature search as well and ways I can
> > reframe the algorithm to find use cases that it is optimal on.
> > >>>
> > >>>
> > >>>
> > >>> I would be curious whether you think the remaining artifacts are
> > more likely caused by inaccuracies in the segmentation boundaries or by
> > the spline approximation of those boundaries? Based on the artifacting
> > you see in the images, I would greatly appreciate any additional
> > suggestions you might have for improving the clarity. I would also be
> > happy to share the code if that would be helpful.
> > >>>
> > >>> Thanks again for all of your insights!
> > >>>
> > >>> Kind Regards,
> > >>> Daniel M. Fisher
> > >>>
> > >>> -----Original Message-----
> > >>> From: Chris Town <cpt23@cam.ac.uk>
> > >>> Sent: Sunday, February 1, 2026 14:33
> > >>> To: D.M. Fisher <dmf41@cantab.ac.uk>
> > >>> Subject: Re: Revisiting 2021 Computer Vision Project for Possible
> > >>> Publication
> > >>>
> > >>> Hi Daniel,
> > >>>
> > >>> Thanks for the update. I hope the cold spell is not causing too many
> > >>> problems, and I am glad that you have been able to make
> progress. Here
> > >>> in Cambridge we are mainly blessed with lots of cold rain, so a
> bit of
> > >>> snowfall would actually brighten things up :-)
> > >>>
> > >>> Well done making those efficiency improvements and fixing the issue
> > with HSV distances! This should make the method work on much larger
> > images and open up many more improvements.
> > >>>
> > >>> Below are some thoughts and suggestions I have at the moment.
> > Please take these only as potential avenues of further work and
> > refinement, they are far from compulsory or exhaustive. Any research
> > project gives rise to an essentially infinite list of "further work", so
> > you need to choose how best to spend your limited time most effectively

> > (when in doubt do whatever seems most fun!).

> > >>>

> > >>> - In terms of colour space, I agree that it is often appropriate to

> > weight H, S, and V channels differently. Human vision is more sensitive

> > to luminance (which is not well approximated by V), and hue H is

> > unreliable (i.e. should be downweighted) when saturation S is low. Also

> > consider CIELAB instead of HSV since the former is designed for

> > perceptual uniformity and Euclidean distances are more meaningful by

> design.

> > >>>

> > >>> - While capping polynomial degree helps, raw polynomials are

> prone to

> > >>> Runge's phenomenon (wild oscillations at edges). That's one

> reason why

> > >>> in our project feedback we suggested things like B-splines, Bézier

> > >>> curves, or Fourier descriptors. These are the standard for

> > >>> applications such as vector graphics because they are numerically

> > >>> stable and locally controllable, unlike global polynomials. You

> should

> > >>> enforce closure by explicitly appending the start point to the end,

> > >>> and validating adjacency constraints before filling regions. You

> could

> > >>> also have a hybrid approach where relatively large/simple

> regions have

> > >>> boundaries encoded as polynomials and use a different method for

> > >>> small/complex region boundaries. If you want to stick to polynomials

> > >>> to differentiate your approach, I suggest piecewise regression

> > >>> (several lower-degree

> > >>> segments) rather than one high-degree curve. Is this already what

> > you're doing when "capping" the polynomials?

> > >>>

> > >>> - In a similar vein, polynomials are continuous functions, but

> > images are discrete grids. When you rasterise a polynomial curve, you

> > effectively quantise it. If two neighbouring regions have polynomial

> > boundaries that do not perfectly "snap" together on the pixel grid, you

> > can get holes or overlaps. You therefore need a robust, deterministic

> > rasterisation/fill rule (e.g. a polygon scanline fill using an even-odd

> > or winding rule). In practice, filling should likely be done by finding

> > the contour, enforcing closure, and then using either (i) a scanline

> > fill or (ii) a flood-fill from seed points with the contour treated as a

> > barrier, rather than calculating all function values.

> > >>>

> > >>> - Rather than relying on exact boundary hits from pointwise

> > plotting, use a deterministic inside/outside test with a clear tie-break

> > rule for boundary pixels (to avoid cracks/gaps between regions). When

> > boundaries do not perfectly close, you can assign ambiguous pixels to

> > the region with the nearest mean colour. You could also add a post-fill

> > connectivity check: as a last-resort robustness step you might use

> > simple morphological operations (e.g. dilation/closing) to seal very

> > small holes, but it's worth keeping this controlled and measuring its

> > impact.

> > >>>

> > >>> - The switch to CCL and proper contour tracing was the right move

> > and addresses the core scalability issue. In order to further reduce

> > runtime complexity, have you considered a multi-scale (coarse-to-fine)

> > approach?

> > >>> For large resolution you could first construct an image pyramid and

> > >>> then run your algorithm on a much lower-resolution subsampled
> version
> > >>> of the image. You would then have to carry the results obtained
> on the
> > >>> lower-resolution image forward to lower levels (higher resolution,
> > >>> less
> > >>> subsampled) of the image pyramid in a way that selectively uses
> > those results (clusters and boundaries) to initialise and guide the
> > segmentation and compression of the finer-grained (higher resolution)
> > images until you apply it to the original full-resolution image.
> > >>> The reasoning is that region topology is largely preserved across
> > scales. A region that exists at full resolution will (usually) still
> > exist at half resolution, just with coarser boundaries. This means
> > coarse-level computation can establish the "big picture" that fine-level
> > computation merely refines. If you would like to try this, then I would
> > recommend Gaussian pyramids rather than simple (binary) subsampling to
> > avoid potential aliasing artefacts. How many levels of subsampling you
> > use is something that would need to be investigated experimentally.
> > >>> Up-sampling the cluster assignment map would require some careful
> > thought; hopefully in most cases only pixels near predicted boundaries
> > would need reconsideration to refine the map. It might be tricky to make
> > it work for topology changes, e.g. merging or splitting regions as
> > needed based on fine-level connectivity, and handling "holes" that only
> > appear in higher-resolution images.
> > >>>
> > >>> - You could even make the pyramid depth adaptive based on some
> > metric of visual complexity. In terms of compression, you could
> > potentially start with a representation of the coarse-scale pyramid
> > level first. This would be especially well-motivated if we consider the
> > benefits of your approach for efficient image transmission (e.g. photos
> > being sent back to Earth from a space probe or Mars rover): you could
> > have progressive encoding/decoding of the different pyramid levels that
> > incrementally adds more detail during transmission and reconstruction.
> > >>>
> > >>> - Parallel processing/GPUs: many operations could be speeded up
> > very significantly on GPUs, and there is also scope to process/refine
> > different regions in different parts of the image in parallel.
> > >>>
> > >>> - Regarding efficient encoding of coefficients, use scaled integers
> > or fixed-point numbers. You should first apply delta coding (storing the
> > difference between the current coefficient/vertex and the previous one)
> > before entropy coding (e.g. Huffman) to drastically reduce entropy. Make
> > sure your final output is an efficient binary format rather than text.
> > >>>
> > >>> - Most polynomial coefficients/coefficient differences for boundary
> > curves will be in predictable ranges, so you could analyse the
> > distribution of coefficients and deltas across your test set and use
> > that to inform the design of your quantisation and encoding, i.e.
> > >>> optimise how the resulting range of values is mapped onto
> 8/16/32 bit
> > >>> fixed-point representations with adaptive scaling. Have a look at
> > >>> entropy encoding methods such as arithmetic coding and ANS
> (Asymmetric
> > >>> Numeral Systems)
> > >>>
> > >>> - Once your method is efficient enough to work on higher-resolution
> > >>> images, consider larger standard datasets that have been used by the

> > >>> research community such as the Kodak dataset
> > >>> <https://r0k.us/graphics/kodak/> <<https://r0k.us/graphics/kodak/>> or
> > <<https://r0k.us/graphics/kodak/>> <<https://r0k.us/graphics/kodak/>>> or
> > BSDS500
> > >>> <https://github.com/BIDS/BSDS500> <<https://github.com/BIDS/BSDS500>>
> > BSDS500> <<https://github.com/BIDS/BSDS500>> <<https://github.com/BIDS/BSDS500>>>
> > >>> Your method will likely work especially well on cartoons/diagrams/
> > graphics/logos etc, so include some of those as well.
> > >>>
> > >>> - To improve performance on images with complex high-frequency
> > information (texture), consider a hybrid approach where you first use
> > your polynomial-based segmentation to encode region-level structure,
> > then subtract the reconstructed image from the original. The difference
> > (reconstruction residuals) could then be compressed separately using a
> > standard method like a wavelet or discrete cosine transform (as in JPEG)
> > at a chosen compression factor/loss level.
> > >>>
> > >>> - Consider comparing (as a baseline for region partitioning
> > quality vs
> > >>> runtime) your method with
> > >>> <https://www.epfl.ch/labs/ivrl/research/slic-superpixels/>
> > <<https://www.epfl.ch/labs/ivrl/research/slic-superpixels/>> <<https://www.epfl.ch/labs/ivrl/research/slic-superpixels/>>
> > <<http://www.epfl.ch/labs/ivrl/research/slic-superpixels/>> <<http://www.epfl.ch/labs/ivrl/research/slic-superpixels/>>>
> > >>> You could even combine this with your approach, i.e. use SLIC as a
> > segmentation initialiser and region proposal step.
> > >>>
> > >>> - You will need some quantitative metrics such as PSNR (Peak
> > Signal-to-Noise Ratio) and SSIM (Structural Similarity Index) to better
> > compare your method vs JPEG/PNG/WEBP, maybe BPG(H.265). Qualitative data
> > could be done using some human evaluators like before, and/or a
> > perceptual metric like LPIPS (Learned Perceptual Image Patch Similarity).
> > >>>
> > >>> - In order to better motivate the work in a publication, think about
> > >>> use cases where non-standard trade-off between data size and
> > >>> reconstruction quality apply. For example, image transmission from
> > >>> probes on the Moon/Mars etc, or image data transmitted from
> > >>> surveillance drones via spread spectrum/UWB low-power covert
> > channels.
> > >>> Another idea would be to frame the approach not just as image
> > >>> compression but as a form of automatic raster-to-vector conversion
> > >>> (e.g. to convert an image into SVG
> > >>> format) and make more explicit comparisons with fractal compression.
> > >>>
> > >>>
> > >>> Best wishes,
> > >>> Chris
> > >>>
> > >>>
> > >>> On 29/01/2026 02:39, D.M. Fisher wrote:
> > >>>> Chris,
> > >>>>
> > >>>> I hope you've been well! We've had a big snow storm on the east
> > coast cancelling work for a few days (perks of still working for the
> > govt) which has given me some time to really dig in to this project.
> > >>>>
> > >>>> I wanted to provide a brief update on the progress I've made on

> > the project since our last discussion.
> > >>>>
> > >>>> As you mentioned in your original feedback, the computational
> > complexity of my original algorithm was immense and led to an inability
> > to test on larger image sizes without partitioning the image. I decided
> > the recursive function that looks for holes had to go and have
> > implemented a 4-connected connected-component labeling (CCL) algorithm,
> > which seems to be working quite well and has allowed me to test on
> > larger images since this significantly cut down the runtime. I have
> > shifted my focus to 50x50 and 256x256 images primarily, enabling more
> > realistic evaluation.
> > >>>>
> > >>>> Testing on larger images has helped me identify several weak
> > points in my algorithm as it was, which I have corrected:
> > >>>> -I was erroneously passing more cluster points than just the edges
> > into the polynomial regression which wasn't noticeable for my small
> > image but is apparent on a larger image and of course causes issues when
> > performing a regression.
> > >>>> -I was sorting clusters based on my inertia algorithm which was
> > far less efficient than doing contour tracing.
> > >>>> -I identified the contour traces weren't always closed loop (in
> > cases that they should have been like the border) which was causing
> > issues in the decompression phase on larger images.
> > >>>> -I was allowing for very high dimensional polynomial fits which
> > wasn't adding much value on larger images, so I have set a max cap on
> > polynomial degree and also zeroed out near-zero coefficients.
> > >>>>
> > >>>> I also identified that I had previously overlooked that in the HSV
> > space, the values wrap once getting past 255, so straight Euclidean
> > distance doesn't make much sense. I have implemented a weighted circular
> > HSV distance instead which has also preliminarily shown improvements in
> > cluster accuracy and color preservation.
> > >>>>
> > >>>> Regarding next steps, I am still having some issues reconstructing
> > the images without significant artifacting during the portion of my
> > algorithm where I fill in gaps that aren't otherwise determined by the
> > initial reconstruction, so I've got more work to do there. Additionally,
> > you mentioned in your feedback that I'm not storing my polynomial
> > coefficients smartly which is hurting my compression ability so I plan
> > to implement quantization and a Huffman encoder to compress the long
> > float coefficients stored by the polynomial regression and run-length
> > encoding for vertical portions of pixels within a cluster.
> > >>>>
> > >>>> At this time I wouldn't say I am stuck on anything in particular,
> > just need to keep working through my script in detail now that it is
> > feasible on larger images and therefore many more test cases are
> available.
> > >>>>
> > >>>> I would greatly appreciate any feedback you might have on these
> > developments thus far or suggestions for further improvements. Thanks!
> > >>>>
> > >>>> Kind regards,
> > >>>> Daniel
> > >>>>
> > >>>> -----Original Message-----
> > >>>> From: Chris Town <cpt23@cam.ac.uk>
> > >>>> Sent: Thursday, January 15, 2026 18:09

> > >>>> To: D.M. Fisher <dmf41@cantab.ac.uk>
 > > >>>> Subject: Re: Revisiting 2021 Computer Vision Project for Possible
 > > >>>> Publication
 > > >>>>
 > > >>>> Hi Daniel,
 > > >>>>
 > > >>>>> out of academia for the past few years. Would you be interested in
 > > >>>>> pursuing this with me? If so, does the timeline of 2-3 months
 > sound
 > > >>>>> reasonable to you?
 > > >>>>
 > > >>>> Yes, that would be suitable. I'm not sure how much time you (or
 > > >>>> indeed
 > > >>>> I) will be able to devote to this project in addition to our main
 > > jobs, but let's give it a good shot. I suggest you email me a progress
 > > update every week or two and let me know if you have particular
 > > questions or issues you'd like my input on.
 > > >>>>
 > > >>>> Best wishes,
 > > >>>> Chris
 > > >>>>
 > > >>>>> Thanks again for your encouragement and guidance, it's been
 > > >>>>> extremely helpful.
 > > >>>>>
 > > >>>>> Kind regards,
 > > >>>>> Daniel
 > > >>>>>
 > > >>>>>
 > > >>>>>
 > -----
 > > >>>>> -
 > > >>>>> -
 > > >>>>> --
 > > >>>>> *From:* Chris Town <cpt23@cam.ac.uk>
 > > >>>>> *Sent:* Monday, January 12, 2026 10:48
 > > >>>>> *To:* D.M. Fisher <dmf41@cantab.ac.uk>
 > > >>>>> *Subject:* Re: Revisiting 2021 Computer Vision Project for
 > Possible
 > > >>>>> Publication Hi Daniel,
 > > >>>>>
 > > >>>>> That sounds like a promising area of research!
 > > >>>>>
 > > >>>>> I suggest you reach out to Geoff Parks <gtp10@cam.ac.uk>, whom I
 > > >>>>> know quite well since he used to be Senior Tutor at Jesus:
 > > >>>>> <https://www.eng.cam.ac.uk/profiles/gtp10> <https://
 > www.eng.cam.ac.uk/profiles/gtp10> <https://
 > www.eng.cam.ac.uk/profiles/gtp10> <http://www.eng.cam.ac.uk/profiles/
 > gtp10>>
 > > >>>>> <https://www.eng.cam.ac.uk/profiles/gtp10 <https://
 > > www.eng.cam.ac.uk/profiles/gtp10>> <http://www.eng.cam.ac.uk/
 > profiles/gtp10>>>
 > > >>>>>
 > > >>>>> Regarding the paper, do keep me updated on progress, although
 > due to
 > > >>>>> time constraints I can realistically not commit to significant
 > > >>>>> involvement, unless you might consider co-authorship.
 > > >>>>>

> > >>>> Best wishes,
> > >>>> Chris
> > >>>>
> > >>>>
> > >>>> On 08/01/2026 15:17, D.M. Fisher wrote:
> > >>>> > Dr. Town,
> > >>>> >
> > >>>> > Thank you very much for your thoughtful response and for
> > >>>> sharing the original feedback, it's extremely helpful to revisit
> > it with fresh eyes.
> > >>>> >
> > >>>> > At this stage, I see this as an interesting project that's
> > >>>> worth revisiting and polishing carefully to help my professional
> > development.
> > >>>> For context, I'm currently a submarine officer in the U.S.
> Navy who
> > >>>> has completed my time at sea and am now in a shore-based role,
> which
> > >>>> has given me the space to deliberately re-engage with
> technical and
> > >>>> research-oriented work in preparation to transition out of the
> navy
> > >>>> and pursue a PhD.
> > >>>> >
> > >>>> > Regarding my PhD interest, my experience on a nuclear
> powered
> > >>>> submarine has given me a strong background in nuclear energy and I
> > >>>> would love to operate at the intersection of computer science and
> > physics.
> > >>>> Specifically, I am interested in using reinforcement learning to
> > >>>> optimize plasma configuration in tokamak control, extending
> some of
> > >>>> the research Deepmind has done in 2022. Do you know of any
> > >>>> colleagues in the computer science or physics departments that may
> > >>>> be interested in this line of work? During my future studies, I'd
> > >>>> love to interface with some of the companies pursuing a successful
> > >>>> fusion reactor such as Commonwealth Fusion Systems (CFS) or
> Helion,
> > >>>> and I know they are also closely tied to MIT's Plasma Science and
> > >>>> Fusion Center and Princeton's Plasma Physics Laboratory. I was
> > >>>> thinking of reaching out to another professor of mine that I
> > worked with quite a bit as well, Dr.
> > >>>> Veličković, who I know works at Deepmind to see if he knows anyone
> > >>>> on the team interfacing with CFS.
> > >>>> >
> > >>>> > Regarding this image compression project, my plan is to
> first
> > >>>> implement the revisions and extensions you suggested. Then, I'd be
> > >>>> very grateful for any feedback you'd be willing to offer at a high
> > >>>> level. I completely understand that your time is limited, and I'm
> > >>>> not expecting ongoing supervision, even brief comments at that
> > stage would be invaluable.
> > >>>> >
> > >>>> > In terms of timescale, I'm aiming to work through the
> > suggested
> > >>>> revisions and extensions over the next two to three months, with a
> > >>>> view toward aligning a submission with one of the venues you

> > >>>> mentioned, depending on how the revisions develop.
> > >>>> > Thanks again for your encouragement and guidance!
> > >>>> >
> > >>>> > Kind regards,
> > >>>> > Daniel
> > >>>> >
> > >>>> >
> > >>>> > -----Original Message-----
> > >>>> > From: Chris Town <cpt23@cam.ac.uk>
> > >>>> > Sent: Tuesday, January 6, 2026 11:42 > To: D.M. Fisher
> > >>>> <dmf41@cantab.ac.uk> > Subject: Re: Revisiting 2021 Computer
> Vision
> > >>>> Project for Possible Publication > > Hi Daniel, > > It is good
> > >>>> to hear from you again! Thank you for your kind words
> regarding John
> > >>>> Daugman.
> > >>>> >
> > >>>> > I am glad that you are considering a PhD. I remember
> you and
> > >>>> your project, and I have also looked up the feedback I/we
> wrote back
> > >>>> in
> > >>>> 2001 (see below) in case it is useful. In terms of publication, I
> > >>>> would recommend first implementing some further revisions and
> > >>>> extensions along the lines of what we suggested. A venue such as
> > >>>> "Journal of Imaging", IET Image Processing, or SPIE "Image
> > Processing: Algorithms and Systems"
> > >>>> > could then be suitable. What time scale do you have in
> mind?
> > >>>> >
> > >>>> > Best wishes,
> > >>>> > Chris
> > >>>> >
> > >>>> > "It is not often that we see an L248 project that tackles
> > image
> > >>>> data at the pixel-level, and we commend you for choosing an
> > >>>> under-appreciated problem with fresh eyes and reasoning about
> it in
> > >>>> some detail. An algorithm that grows with the square of the image
> > >>>> resolution (even when ignoring increasing cluster numbers) is of
> > >>>> course only going to be of theoretical use, but it is good
> that you
> > >>>> thought carefully about how your algorithm should be designed to
> > >>>> cope with different region topologies. It should be noted that
> other
> > >>>> compression schemes sometimes handle troublesome cases by
> splitting
> > >>>> regions (and encoding the parts > separately) or by having a
> > >>>> fall-back compression method (perhaps as simple as run-length
> > >>>> encoding or a dictionary-based approach), which would help to
> reduce
> > >>>> the effective computational requirements of your methods.
> > >>>> > To allow a more real-world usage case, it would have been
> > >>>> interesting to apply your method to separate 50x50 pixel (or
> > >>>> whatever block size is > appropriate) blocks taken from larger
> > >>>> images. This would have enabled you to conduct a more thorough
> > >>>> evaluation, since the comparison with JPEG and PNG for 50x50

> images
> > >>>> shown in your report is somewhat biased against the overheads of
> > >>>> those techniques
> > >>>> (50x50 in particular is not a good match for JPEGs block
> size). Even
> > >>>> just using
> > >>>> 50x50 images, we would really have liked to have seen a
> > >>>> significantly larger number of test cases to better gauge the
> > performance of and drawbacks your algorithms.
> > >>>> > Given the number of bytes required to represent accurate
> > >>>> floating point data, the set of images on which your algorithm is
> > >>>> likely to yield a reduction in data storage is further limited,
> > >>>> although you could mitigate this using fixed-point representations
> > >>>> and additional compression of your coefficient data.
> > >>>> > Lastly, given the scope of a mini-project, one cannot
> expect a
> > >>>> truly comprehensive literature survey. However, given the topic of
> > >>>> the report, we would have liked to have seen at least some mention
> > >>>> of fractal image compression and the theoretical limits if
> > >>>> compressibility, such as Kolmogorov complexity. While there is
> some
> > >>>> novelty in the approach taken in your report, you could have
> > >>>> mentioned related work that makes use of techniques such as
> > >>>> polynomial B-splines, trigonometric Bezier Curves, and wavelets to
> > >>>> encode localised approximations to image content. If you did
> want to
> > >>>> take this work further, we would also recommend that you look
> into a
> > >>>> wider range of colour image segmentation methods (e.g. using a
> > >>>> combination of edge finding, Delaunay triangulation and
> > agglomerative clustering).
> > >>>> > "
> > >>>> >
> > >>>> > On 06/01/2026 02:57, D.M. Fisher wrote:
> > >>>> >> Dear Dr. Town,
> > >>>> >>
> > >>>> >> I hope you've been well. I was very sorry to hear of
> > Professor
> > >>>>> Daugman's passing. He was an inspiring, charismatic instructor,
> > >>>> and I
> > >>>>> learned so much from him in the Computer Vision course.
> > >>>> >>
> > >>>> >> I completed my MPhil in ACS in 2021, where you and
> Professor
> > >>>> Daugman >> were my course instructors for CV, along with Dr.
> > Mahmoud.
> > >>>> Recently, I >> was revisiting some older work and came across my
> > >>>> final project from >> the course on segmentation-based image
> > >>>> compression using polynomial >> regression. I greatly appreciated
> > >>>> both your feedback and Professor >> Daugman's, and I recall
> that he
> > >>>> mentioned the work might be worth >> considering for publication
> > with some refinement.
> > >>>> >>
> > >>>> >> With some distance from it now, I'm considering whether it
> > >>>> might be >> worth revisiting and polishing for submission,
> > >>>> particularly as I've >> been exploring PhD programs recently.

> I've
> > >>>> tentatively been looking at >> venues such as the /International
> > >>>> Journal of Computer Vision/, as well >> as /IEEE Transactions on
> > >>>> Image Processing/, but I wanted to ask >> whether you think this
> > >>>> project still seems worthwhile and if you'd >> recommend any
> > particular venue or reframing.
> > >>>> >>
> > >>>> >> I've recently gone back and made some corrections to the
> > >>>> original
> > >>>>>> paper and attached the revised version for context. I completely
> > >>>>>> understand if you don't have time for a detailed review,
> even >>
> > >>>> high-level guidance on suitability, positioning, or venue choice
> > >>>> would be extremely helpful.
> > >>>> >> Thanks!
> > >>>> >>
> > >>>> >> Kind regards,
> > >>>> >> Daniel Fisher
> > >>>> >>
>